

SYSTEMATISK STRATEGIFORSKNING · TRADING-AGENT

Dammluckan

Ockupationsbetingad rekordhasard

En förregistrerad falsifieringsstudie på multi-asset-ETF:er, 2004-2026

VERDIKT: FÖRKASTAD

Universum 16 multi-asset-ETF:er (primär) + 16 lands-ETF:er (OOS-2)

Data EODHD, dagliga OHLCV, 2003-01-01 – 2026-08-07, PIT

Kod 20 Python-filer, 2405 rader, 30 tester

Branch `claude/dammluckan-record-hazard-x2qjhq`

Innehåll

Del I — Resultat

1. Forskningsrapport (REPORT.md) — hypotes, grindar, nollbatteri, verdikt
2. Arkitektur och deklarerade avvikelser (README.md)
3. Resultatartefakter — 27-cells grid och sammanfattande JSON

Del II — Källkod (20 filer, 2405 rader)

1. `config.py` — Deklarerade konstanter: universum, IS/OOS-gränser, grid, kostnadshinkar, dödsriterier. (134 rader)
2. `fetch_data.py` — EODHD-hämtning med JSON-cache; skriver OHLCV-paneler till data/. (107 rader)
3. `data.py` — Panel-dataklass: justerad OHLC-rekonstruktion, dollarvolym, avkastning, vol, ADV. (131 rader)
4. `signal.py` — Kärnsignalen: M_t , $E\pm$, $O\pm$ samt kalibrering av bandkonstanten c och θ_i . (179 rader)
5. `nulls.py` — Cirkulär blockbootstrap: primitiver för θ/c -kalibrering och Steg-1-nollan. (87 rader)
6. `costs.py` — ADV-hinkad transaktionskostnadsmodell (porterad från Formdriften). (51 rader)
7. `portfolio.py` — Positions-/trade-primitiver, invers-vol-vikt, rundturskostnad. (60 rader)
8. `backtest.py` — Händelsedrivna motor: kandidater, FIFO-admission, sizing, dagsavkastning, IS/OOS. (259 rader)
9. `twins.py` — Donchian-tvilling (ogated) och anti-tvilling (låg ockupation). (38 rader)
10. `tsmom.py` — TSMOM-proxy för redundans- och diversifieringskontroll. (72 rader)
11. `battery.py` — Nollhypotes #3 (blockpermuterade kurvor) och #4 (slumpade entrytidpunkter). (146 rader)
12. `robustness.py` — Steg 1: händelsenivå-IC, IC-null och redundansscreen mot 6 kontrollvariabler. (169 rader)
13. `metrics.py` — Sharpe, DSR/PSR, Newey-West, IC, PnL-koncentration, teckenstabilitet. (147 rader)
14. `grid.py` — 27-cells grid, DSR-deflation och grannskapets teckenmajoritet. (86 rader)
15. `run_calibration.py` — Steg 0-förberedelse: c och θ_i , IS-fryst till output/calibration.pkl. (66 rader)
16. `run_research.py` — Full pipeline-orkestrering, cache:ad per stadium. (252 rader)
17. `tests/test_signal.py` — Kausalitet/no-look-ahead, ockupationsgränser, monoton i c . (96 rader)
18. `tests/test_backtest.py` — Exit-prioritet, FIFO-admission, handberäknad dagsavkastning, bruttotak. (218 rader)
19. `tests/test_costs.py` — ADV-hinkgränser och handberäknad kostnadsfraktion. (34 rader)
20. `tests/test_metrics.py` — Sharpe, DSR-kalibrering, koncentration, teckenkonsistens. (73 rader)

Del I — Resultat

Forskningsrapporten i sin helhet, följd av arkitektturnoteringar och råa resultatartefakter.

Dammluckan — ockupationsbetingad rekordhasard: resultat

Verdikt: FÖRKASTAD. Hypotesen faller på tre av fyra förregistrerade dödsriterier: Steg 0 (händelseräkning, kort sida), nettoöverskott mot Donchian-tvillingen (OOS) och DSR (OOS). Steg 1 (estimatornull) klaras dock tydligt — ockupationen O^+ har en genuin, icke-redundant prediktiv koppling till framtida avkastning på händelsenivå. Det denna prediktiva kraft inte gör är att slå den naiva, redan kända Donchian-projektionen efter att portföljen byggts med rimlig FIFO-kapacitet och volmålsättning: den mest sannolika dödsorsaken som förregistrerades i brief:en — "blir strategin oskiljbar från Donchian-tvillingen" — är exakt vad som hände.

0. Infrastruktur och universum

Detta repo har ingen enda delad kodbas för PIT-data/backtest — varje tidigare forskningsgren (Vridmomentet, Vindkastet, Formdriften, Fasflocken, Oglegrinden, Irreversibility Lab) är ett självständigt paket på sin egen branch, med starkt liknande men inte identiska konventioner. Dammluckan återanvänder medvetet tre specifika stycken av den historiken snarare än att uppfinna nytt:

- **Universum:** Vindkastets exakta primära 16-ETF-panel (SPY, IWM, EFA, EEM, TLT, IEF, LQD, HYG, GLD, SLV, DBC, USO, UUP, FXE, FXY, VNQ — aktieindex/duration/krediter/guld/olja/bred råvara/USD-proxy/EM, i linje med brief:ens kategorilista) och Vindkastets exakta sekundära 16-landsETF-panel (EWJ, EWG, EWU, EWQ, EWI, EWP, EWL, EWA, EWC, EWY, EWT, EWZ, EWW, EWS, EWH, EWD). **Deklarerad avvikelser:** brief:en skriver "20" multi-asset-ETF:er; Vindkastets 16-namnspanel är den enda existerande EODHD-panel i repot som matchar brief:ens tillgångsklasskategorier, och återanvänds hellre än att en ny 20-panel konstrueras ad hoc.
- **Data:** EODHD ([EODHD_API_KEY](#) i miljön, fungerande i denna sandbox — till skillnad från flera systerbranscher som fick falla tillbaka på Yahoo Finance). PIT-hantering: råstängningskurs används för rekord-/ockupationsdetektion (se §2 nedan för motivering), justerad stängning/öppning för avkastningsberäkning, ingen framåtfyllning över luckor.
- **ADV-hinkmodellen:** Formdriftens [costs.py](#) (den enda ADV-hinkade kostnadsmodellen i hela seriens historik — övriga grenar använder platt commission+half-spread) återanvänd verbatim, inklusive den dokumenterade spread-caveaten (ingen quoterad spreaddata från EODHD; half-spread approximeras via ADV-hinkar, endast commission-benet på 3bp/sida är en ren, icke-approximerad siffra).
- **TSMOM-proxyn:** Formdriftens [tsmom.py](#)-konstruktion (12-månaderstecken, invers-63d-vol, 15%-tak, gross-åternormaliserad) återanvänd med samma exekveringskonvention.
- **DSR/PSR:** Bailey & López de Prado-formeln, kopiestructurellt identisk över Fasflocken/Oglegrinden/Irreversibility Lab/Formdriften; återanvänd här rakt av.

Ingen befintlig gren tillhandahöll dock en händelsedrivna, FIFO-kapad, överlappande flerdagspositionsmotor — alla systerstrategier är enda-bok, full-ersättnings vecko-/

månadsrebalanseringar. Backtest-motorn ([backtest.py](#) , [portfolio.py](#)) är därför byggd från grunden för Dammluckan.

1. Uppsättning som faktiskt kördes

- **Universum:** primär 16-ETF-panel (ovan), IS = 2004-01-01-2017-12-31, OOS-1 = 2018-01-01-2026-08-07. Data från 2003-01-01 används som uppvärmningsbuffert för n=160-fönstrets rullande lookback.
- **Signal:** M_t/m_t = rullande max/min av rå stängningskurs över $[t-n, t-1]$ (exkluderar t). $E^+_t = 1\{P_t > M_t\}$, $E^-_t = 1\{P_t < m_t\}$. O^+_t/O^-_t = andel dagar i samma fönster inom bandet $c \cdot \hat{\sigma} \cdot \sqrt{5}$ från taket/golvet, $\hat{\sigma}$ = 60-dagars realiserad volatilitet av dagliga logavkastningar (kausal, $t-1$). c kalibrerades **per fönsterlängd n** via bisektion på en blockbootstrap-null (blocklängd 20 dagar, cirkulär) tills nollmedianen för $O^+ \approx 15\%$: $c(n=80)=0,996$, $c(n=120)=1,309$, $c(n=160)=1,543$.
- **θ_i :** 80:e percentilen (grid: 70/80/90) av tillgångens egen blockbootstrap-nollfördelning för O^+/O^- , skattad IS (2004-2017), fryst för OOS. Primärcellens θ_i (n=120, p80) spänner 0,225-0,333 (lång) och 0,200-0,258 (kort) över de 16 tillgångarna.
- **Primärcell:** n=120, θ -percentil=80, h=10 handelsdagar.
- **Deklarerad grid:** $n \in \{80, 120, 160\} \times \theta\text{-percentil} \in \{70, 80, 90\} \times h \in \{5, 10, 15\} = 27$ celler, samtliga körda, DSR-deflaterade mot primärcellen.
- **Exit:** h-dagars time stop eller motsatt rekord (oavsett ockupation), vilket som först. I praktiken avgörs **443 av 443 primärcellstrades av time stop och endast 1 av opposite-record** — $n \gg h$ gör att en fullständig rekordreversering inom hållperioden är ett extremt sällsynt utfall, ett rent empiriskt fynd, inte en bugg (verifierat över hela griden: opposite-record-andelen är 0 för alla celler utom de två minsta n/största h-kombinationerna, där den ändå bara är 3-12 av flera hundra).
- **Sizing:** $w_i = k/\hat{\sigma}_i$ ($\hat{\sigma}_i$ = 20-dagars annualiserad realiserad vol, kausal $t-1$), k löst iterativt (inte enstegs-linjärt, se §7) så att IS-realiserad portföljvol $\approx 8\%$ årligen. Max 12 samtidiga positioner (FIFO: redan öppen position i samma tillgång ignorerar nya händelser; en blockerad ny händelse vid fullt kapacitetstak **förkastas**, köas inte). Brutto $\leq 200\%$ (mjuk gräns: ny position hårdklipps ner till kvarvarande utrymme, befintliga positioner ändras aldrig i efterhand).

2. En metodologisk precisering: rå kontra justerad kurs

Rekord-/ockupationsdetektion (M_t , E^+/E^- , O^+/O^-) körs på **rå (ojusterad) stängningskurs**, inte utdelningsjusterad. Motivering: hypotesen handlar om ankring till faktiska historiska handlade prisnivåer (vilande säljlimiter parkerade vid bokstavliga gamla toppar) — en utdelningsjusterad serie skulle tyst flytta dessa historiska nivåer för högavkastande namn (HYG, LQD, TLT, VNQ, EEM ...) och förvränga exakt den mekanism som testas. Allt nedströms om entrybeslutet (avkastning, volatilitet, sizing, Sharpe) använder den justerade serien, standardkonventionen för totalavkastningsmätning.

3. Steg 0 — händelseräkning och koncentration

Sida	Händelser (θ -gated, hela samplet)	Krav	Resultat
Lång (E^+ , $O^+ \geq \theta$)	479	≥ 300	PASS
Kort (E^- , $O^- \geq \theta$)	254	≥ 300	FALL
Koncentration lång	12,1 % (SPY)	≤ 50 %	PASS
Koncentration kort	10,2 % (SLV)	≤ 50 %	PASS

Kortsidan klarar inte det förregistrerade minimikravet på 300 händelser. Detta är inte en gränssfallsskillnad (254 mot 300, -15 %) och inte en koncentrationsartefakt — händelserna är väl spridda över alla 16 tillgångar (mellan 8 och 26 per tillgång). Det är en genuin frekvensbrist: nedgångsrekord med hög ockupation nära golvet är helt enkelt sällsyntare än uppgångsrekord med hög ockupation nära taket i detta 23-åriga multi-asset-sample, konsekvent med universumets sekulära uppåtlutning (aktier och guld i stora, ihållande bull-regimer; obligationers golvbrott är sällan lika "ockuperade" som aktiers takbrott). **Per protokollet är detta ensamt tillräckligt för att förkasta den spegelvända (båda-sidor) hypotesen som specificerad.** Resten av batteriet körs och redovisas ändå, i sin helhet, för transparens — i linje med seriens konvention att en fallerad grindkontroll inte är en anledning att sluta mäta.

4. Steg 1 — estimatornull (före portföljbygge)

Test	Resultat	Krav	Resultat
Händelsenivå-IC (O mot h-dagars signerad forward return, poolat lång+kort)	0,0692 (n=8842 händelser)	$ IC \geq 0,03$	PASS
IC-nollhypotes (blockbootstrap, 500 dragningar)	p=0,0060	p<0,05	PASS
Redundansscreen (poolad OLS R^2 av O mot vol/skew/ r -autokorr/n-dagarsmomentum/avstånd-till-max/TSMOM-proxy)	$R^2=0,0056$ (n=8611)	$R^2 \leq 0,50$	PASS

Steg 1 klaras tydligt på alla tre delar. Spearman-korrelationerna mellan O och varje enskild kontrollvariabel är också små (vol 0,031, skew $-0,012$, |r|-autokorr $-0,017$, n-dagarsmomentum 0,102, avstånd-till-max 0,146, TSMOM-proxy 0,082) — ockupationen är inte en förklädd version av någon av dessa. **Detta är den mest intressanta positiva upptäckten i hela studien:** vid själva händelsetillfället bär O^+/O^- genuin, statistiskt signifikant, icke-redundant information om vad som händer under de följande h dagarna. Frågan är om den informationen överlever kontakten med en faktisk portfölj — se §5.

5. Huvudresultat: primärcell mot Donchian-tvilling mot anti-tvilling

Strategi	IS Sharpe	OOS-1 Sharpe	Full Sharpe	Full max DD	n trades	Genomsn. samtidiga positioner
Dammluckan (primär)	0,303	0,420	0,347	−22,3 %	443	0,74
Donchian-tvilling (ogated)	0,287	0,448	0,363	−14,4 %	2430	4,09
Anti-tvilling (låg ockupation)	−0,107	−0,168	−0,113	−37,2 %	987	—

Nettoöverskott mot Donchian-tvillingen: **IS +0,016, OOS-1 −0,028**. Tecknet flippas mellan delperioderna och båda talen är i praktiken brus — detta är "oskiljbar", inte "sämre" i någon dramatisk mening, men det är fortfarande **≤ 0 på OOS-1**, vilket per protokollet är ett dödsriterium.

Mekanismen bakom, styrkt av handelsdata: ockupationsvillkoret identifierar individuellt bättre affärer (genomsnittlig bruttoavkastning per trade 0,55 % mot Donchians 0,26 %, hit rate 55,5 % mot 53,3 % — konsekvent med Steg 1:s positiva IC), men gör det genom att filtrera bort ~82 % av alla brott (443 av 2430 kvalificerande händelser). Konsekvensen: primärstrategin har i genomsnitt bara 0,74 samtidiga positioner (mestadels kontant, max 7 av 12 platser), mot Donchians 4,09 (ofta nära taket på 12). För att båda ska nå samma 8-procentiga volmål måste den underdiversifierade primärstrategin storleksätta varje enskild position mycket aggressivare (genomsnittlig absolut vikt 65,8 % mot Donchians 26,9 %). Diversifieringsvinsten av att hålla fyra gånger så många okorrelerade positioner samtidigt åter upp — och något mer än så på OOS-1 — den per-trade-fördel som ockupationsvillkoret faktiskt identifierar. Anti-tvillingens tydligt negativa Sharpe (både IS och OOS-1) bekräftar dock att ockupation inte är irrelevant: relationen mellan ockupation och framtida avkastning är påtaglig från "låg" till "medel" (Donchian, obetingad) — men **planar ut** mellan "medel" och "hög" ($\theta \geq 80$:e percentilen) snarare än att fortsätta stiga monotont, vilket är en svagare form av brief:ens $\beta > 0$ -prediktion än vad som förutsattes.

Newey-West t-stat för medelavkastningen: 1,89 (full sample), 1,40 (OOS-1) — i linje med en positiv men inte starkt signifikant absolut avkastning, konsekvent med bilden ovan.

6. Nollhypotesbatteri

Nolltest	Metod	Resultat
(3) Blockpermuterade avkastningar per tillgång	Hela regelverket (frys $n/c/\theta_i/h/k$) på 500 syntetiska, oberoende blockbootstrappade priskurvor per tillgång (blocklängd 20d, cirkulär), bevarar varje tillgångs egen NaN-prefix (inträdesdatum)	Real periodisk Sharpe 0,0219 mot nollmedelvärde $-0,0075 \pm 0,0118$. p=0,0100
(4) Slumpade entrytidpunkter	Samma antal trades per (tillgång, riktning) som verklig admitterad mängd, slumpade entrydatum, samma exit-/sizing-/portföljregler, 500 dragningar	Nollmedelvärde $0,0009 \pm 0,0122$. p=0,0499

Strategins **absoluta** avkastningsstruktur är alltså inte statistiskt oskiljbar från ren brus — den slår båda nollhypoteserna, om än nollhypotes (4) med minsta möjliga marginal ($p=0,0499$, precis under 0,05). Detta är dock en svag måttstock: Donchian-tvillingen, som har högre Sharpe än primärstrategin på OOS-1, skulle sannolikt klara samma nollhypoteser ännu tydligare. "Bättre än brus" är inte samma sak som "bättre än det enklaste redan kända alternativet" — och det senare, inte det förra, är den relevanta ribban för kapitalallokering.

7. DSR-deflaterad grid (27 varianter)

Se [output/grid_table.csv](#) för alla 27 celler. Primärcellen (n120_p80_h10) rankas som **14:e av 27** på OOS-1-Sharpe — långt ifrån bäst i sin egen förregistrerade grid (bästa cellen, n160_p80_h10, når OOS-1-Sharpe 0,843, mer än dubbelt så högt).

	DSR-sannolikhet	DSR-överskott (Sharpe-skala)
IS	0,441	−0,0024
OOS-1	0,384	−0,0062

Båda negativa → **dödsriteriet "DSR ≤ 0 OOS" utlöses**. Marginalen är liten (DSR-sannolikheten 0,38 är inte extremt låg, bara klart under vad som skulle krävas för signifikans efter multipel-testning-avdrag), men riktningen är entydig: primärcellen är inom sin egen 27-cells provpool statistiskt oanmärkningsvärd.

Granskaps-teckenmajoritet (delar primärcellens tecken bland de 6 celler som skiljer sig i exakt en axel): IS 83 % (5/6), OOS-1 100 % (6/6) — **PASS**. Den positiva Sharpe-riktningen är alltså robust över granskandet (ingen isolerad lyckoträff), men det räddar inte cellen från att vara medioker i absoluta DSR-termer.

8. Diversifiering och stabilitet

Kriterium	Resultat	Krav	Resultat
ρ (TSMOM), full sample	0,050	$\leq 0,4$	PASS (brett marginal)
ρ (TSMOM), OOS-1	0,009	$\leq 0,4$	PASS (brett marginal)
Teckeninkonsistens, 4 lika stora delperioder (2004–2026)	1 av 4 (2009-08–2015-04 negativ, övriga tre positiva, helprovtecken positivt)	<2 av 4	PASS
Max kvartals-PnL-andel	4,2 %	Redovisas (ingen hård gräns i brief:en)	Mycket lågt — inte en mars-2020-historia

Strategin är alltså **inte** en förtäckt TSMOM-exponering och **inte** koncentrerad i ett fåtal episoder — de två svagaste förhandsspekulerade dödsorsakerna ($\beta_{\text{TSMOM}} > 0,4$ respektive PnL-koncentration) materialiserades inte alls. Det var Donchian-tvilling-kriteriet och DSR som avgjorde utfallet, precis som brief:en själv rankade som mest sannolikt.

9. OOS-2 — sekundäryta (16 landsETF:er, en enda konfirmationskörning)

Körd exakt en gång, sist, med n/c/h/k fryst verbatim från primäruniversumets IS-kalibrering; endast θ_i omkalibrerades (samma protokoll: 80:e percentilen av tillgångens egen IS-blocknull, eftersom θ_i per definition är tillgångsspecifik och landsETF:erna är andra tillgångar).

Dammluckan		Donchian-tvilling
Sharpe (hela samplet)	0,252	0,084
n trades	514 (683 lång / 226 kort)	—
Annualiserad vol	10,8 % (avvikelse från 8 %-målet väntad — k är transplanterad oförändrad, inte omkalibrerad för denna ytas volregim)	—

Här slår Dammluckan sin egen Donchian-tvilling tydligt (nettoöverskott +0,168) — **motsatt mönster mot primäruniversumet**. Detta rapporteras ärligt utan att tona ner primäruniversumets negativa utfall: en enda konfirmationskörning på en annan tillgångsklass som pekar åt andra hållet är inte en frikännande — den ger en spretig, inkonsekvent replikeringsbild snarare än en tydlig bekräftelse eller en tydlig ytterligare falsifiering. Kort-sidans händelseräkning (226) hade också fallit på samma Steg-0-tröskel om den hade tillämpats här (den tillämpas inte, per protokoll — detta är en konfirmationsyta, inte ett eget förregistrerat test), vilket är en relevant kontext snarare än en friskrivning.

10. Dödskriterier — sammanfattning

Kriterium	Resultat	Verdikt
Steg 0 (händelseräkning ≥ 300 /sida)	Kort sida: 254	FALL
Steg 0 (koncentration ≤ 50 %/tillgång)	Max 12,1 %	PASS
Steg 1 ($ IC \geq 0,03$ och $p < 0,05$)	$IC = 0,069$, $p = 0,006$	PASS
Steg 1 (redundans $R^2 \leq 0,5$)	$R^2 = 0,006$	PASS
Nettoöverskott mot Donchian-tvilling ≤ 0 (OOS-1)	-0,028	FALL
DSR ≤ 0 (OOS-1)	-0,0062	FALL
Teckeninkonsistens ≥ 2 av 4 delperioder	1 av 4	PASS
$\rho(\text{TSMOM}) > 0,4$	0,009 (OOS-1)	PASS

Tre av åtta kontroller, varav två (Donchian-överskott och DSR) explicit förregistrerade som primära dödsriterier, utlöser förkastande.

11. Var hypotesen går sönder

Brief:ens egen rankning av mest sannolika svaghet var: "betingningen adderar inget — hög ockupation är delvis 'låg vol före brott', och efter redundanskontroller blir strategin oskiljbar från Donchian-tvillingen, dvs. sämre trendföljning." Det som faktiskt hände är en precisering, inte en enkel bekräftelse:

- Betingningen adderar **inte ingenting** — Steg 1:s IC-test och redundansscreen visar tydligt att O^+/O^- bär genuin, icke-redundant prediktiv information vid händelsetillfället (detta var den förhandsspekulerade "mest sannolika döden" som inte inträffade).
- Men den vinsten äts upp av ett kapacitetspris: genom att bara handla den mest "ockuperade" femtedelen av brotten tappar strategin fyra femtedelar av sin diversifieringsbredd. Vid lika volmål slår den förlorade diversifieringen den vunna per-trade-precisionen, netto till strategins nackdel på OOS-1.

- Anti-tvillingens tydligt negativa avkastning visar att ockupation är informativ i stort (låg ockupation → dåligt), men förhållandet är mer troskel-/stegformat (dåligt → medel → medel) än den kontinuerligt stigande dos-respons-relation som brief:en implicit förutsatte (medel → allt bättre med allt högre ockupation).
- Den näst mest sannolika förhandsspekulerade döden ($\beta_{TSMOM} > 0,4$) och den tredje (PnL-koncentration i få episoder) materialiserades inte alls — diversifieringen mot känd trendföljning är utmärkt och avkastningen är brett spridd över tid.
- Steg 0:s kortsidesbrist är en oberoende, strukturell datafrekvensbegränsning (nedgångsrekord med hög golv-ockupation är sällsynta i detta universum/sampel) snarare än ett metodfel — den skulle sannolikt kvarstå även med andra (n, θ)-val inom griden, givet att ingen av de 27 cellernas kortsidesräkningar redovisas separat men den underliggande händelsefrekvensasymmetrin (lång 479 mot kort 254 på primärcellen) är strukturell, inte en artefakt av just denna cells parametrar.

12. Slutsats

Dammluckan-hypotesen förkastas som specificerad. Det finns ett genuint, statistiskt robust samband mellan ockupation och efterföljande avkastning vid själva rekordhändelsen (Steg 1), men det samband överlever inte kontakten med en riktig, kapacitetsbegränsad, volmålsatt portfölj som ska konkurrera mot sin egen obetingade Donchian-projektion. Resultatet utesluter inte att en annan portföljkonstruktion (mindre restriktiv gating, kombinerad med snarare än ersättande av bredare trendföljning, eller en sizing-regim som inte kräver lika hög diversifiering för att nå volmålet) skulle kunna återvinna en del av den identifierade per-trade-fördelen. Det utesluter inte heller att fenomenet är verkligt på andra tillgångsklasser (OOS-2:s motsatta resultat antyder det, om än på en enda, icke-förregistrerad-som-avgörande körning). Vad resultatet utesluter är den exakta, förregistrerade regeln som brief:en specificerade, körd på det universum den specificerade.

13. Reproducerbarhet

```
cd research/dammluckan
pip install -r requirements.txt
export EODHD_API_KEY=... # krävs för fetch_data.py; redan cachead under data/
python -m research.dammluckan.fetch_data # (redan körd; data/*.csv committade)
python -m research.dammluckan.run_calibration # c, theta_i per (n, sida, percentil) -> output/calibration.pkl
python -m research.dammluckan.run_research # Steg 0/1, primär+tvillingar+nollor, grid+DSR, OOS-2
pytest -q # 30 tester
```

Alla siffror i denna rapport är reproducerade från `output/results_summary.json` och `output/grid_table.csv`. Nollbatteriernas 500 dragningar (`config.N_BLOCK_DRAWS`) och θ_i -kalibreringens 1000 dragningar (`config.N_THETA_CALIB_DRAWS`) är deklarerade konstanter, inte tunade i efterhand.

Filer: `config.py` (deklarerade konstanter) · `data.py / fetch_data.py` (EODHD-panel, PIT) · `signal.py` (M_t/E/O, θ -kalibrering) · `nulls.py` (blockbootstrap-primitiv) · `backtest.py / portfolio.py` (händelsedrivna motor, FIFO/bruttotak, volmålsättning) · `costs.py` (ADV-hinkmodell) · `tsmom.py` (TSMOM-proxy) · `twins.py` (Donchian/anti-tvilling) · `battery.py` (nollor #3/#4) · `robustness.py` (Steg-1 IC-null + redundans) · `metrics.py` (Sharpe/DSR/PSR/NW/IC) · `grid.py` (27-cells grid, DSR, grannskap) · `run_calibration.py / run_research.py` (orkestrering) · `tests/` (30 tester).

Arkitektur

Modulöversikt, återanvänt material från systergrenar och deklarerade avvikelser från brief:en.

Dammluckan — ockupationsbetingad rekordhasard

Se [REPORT.md](#) för resultat och verdikt (FÖRKASTAD). Den här filen beskriver arkitekturen.

Hypotesen

Deltagare ankrar order vid historiska extremer: limitförsäljningar vid gamla toppar (dispositionseffekten), stopp under gamla bottnar. Det deformerar rekordhasarden — sannolikheten att ett nytt pris slår fönstrets max/min — relativt slumpvandringens (volatilitetsfria, Sparre Andersen/arcsinuslag-) null: undertryckt strax under taket, förhöjd efter brott, och förhöjningen växer med hur mycket ordermassa (mätt som ockupationstid nära taket/golvet) som absorberats innan brottet.

- M_t = rullande max/min av rå stängningskurs över de föregående n dagarna (exkluderar dagens kurs).
- $E^+_t / E^-_t = 1$ om dagens kurs slår ett nytt max/min mot M_t/m_t .
- O^+_t / O^-_t = andel av de senaste n dagarna som legat inom ett smalt band (kalibrerat i volatilitetsenheter) från taket/golvet — ett mått på hur mycket "vilande ordermassa" som troligen låg parkerad där.
- θ_i = en tillgångsspecifik tröskel på O , satt som en percentil av tillgångens egen nollfördelning (blockbootstrap), skattad in-sample och fryst out-of-sample. Perentiltröskeln garanterar per konstruktion en icke-tom händelsemängd, oavsett tillgångens egen geometri — en direkt lärdom från en tidigare gren (Vindkastet) vars fasta, icke-kalibrerade absoluta tröskel producerade ett i praktiken tomt händelseset.

Vem förlorar: ankrade innehavare som säljer vid gamla toppar och stopp-placerare under gamla bottnar — deras vilande order utgör väggen som absorberas vid ett genuint brott.

Arkitektur

Ingen tidigare gren i det här repot tillhandahöll en händelsedriven, FIFO-kapad flerdagspositionsmotor med överlappande positioner — alla systerstrategier (Vridmomentet, Formdriften, Oglegrinden ...) är enda-bok, full-ersättnings vecko-/månadsrebalanseringar. Motorn nedan är byggd från grunden för Dammluckan; allt annat (universum, kostnadsmodell, TSMOM-proxy, DSR-formel) är medvetet återanvänt från tidigare grenar — se [REPORT.md](#) §0 för exakt vad som kommer varifrån.

```
research/dammluckan/
├─ config.py          deklarerade konstanter (universum, grid, kostnader, dödsriterier)
├─ fetch_data.py      EODHD-hämtning + JSON-cache -> data/{primary,secondary}<field>.csv
├─ data.py            Panel-dataklass: justerad OHLC-rekonstruktion, dollarvolym, avkastning, vol
├─ signal.py           $M_t/E \pm 0 \pm$ ,  $c$ - och  $\theta_i$ -kalibrering (blockbootstrap-null, bisektion)
├─ nulls.py           cirkulär blockbootstrap: primitiver för  $\theta/c$ -kalibrering + Steg-1 IC-null
├─ costs.py           ADV-hinkkostnadsmodell (ported från Formdriften)
├─ tsmom.py           TSMOM-proxy (ported från Formdriften)
├─ portfolio.py/backtest.py händelsedriven motor: entry  $t+1$  open, exit  $h$ -dagars stop/motsatt rekord,
```

	FIFO/max-12-samtidiga, invers-vol-sizing, bruttotak 200%, iterativ volmålskalibrering (robust mot att bruttotaket binder)
— twins.py	Donchian-tvilling ($\theta=-\infty$, ogated) och anti-tvilling (låg ockupation)
— battery.py	nollhypotes #3 (blockpermuterade priskurvor, hela regelverket) och #4 (slumpade entrytidpunkter, matchat antal/tillgång)
— robustness.py	Steg-1: händelsenivå-IC + null, redundansscreen mot 6 kontrollvariabler
— metrics.py	Sharpe/DSR/PSR (Bailey & López de Prado)/Newey-West/IC/koncentration
— grid.py	27-cells grid ($n \times \theta$ -percentil $\times h$), DSR-deflation, grannskapskontroll
— run_calibration.py	Steg 0-förberedelse: c och θ_i , IS-fryst -> output/calibration.pkl
— run_research.py	full pipeline-orkestrering, cache:ad per stadium
— tests/	30 pytest, se nedan

Deklarerade avvikelser från brief:en

- **Universum:** "≈20" multi-asset-ETF:er tolkas som Vindkastets exakta 16-namnspanel (nästan, inte exakt 20 — men den enda existerande matchande EODHD-panelen i repot; se [REPORT.md](#) §0).
- **Signalkurs:** rekord-/ockupationsdetektion körs på RÅ (ojusterad) stängningskurs, inte utdelningsjusterad — se [REPORT.md](#) §2 för motivering. Avkastning/vol/Sharpe använder justerad kurs.
- **Bandkonstanten c:** kalibrerad separat per fönsterlängd n (inte en global konstant), var och en mot samma 15%-nollmedian-mål.
- **$\hat{\theta}$ i ockupationsbandet:** 60-dagars realiserad vol av dagliga logavkastningar, kausalt fördröjd en dag (uses data through $t-1$, inte t) för att undvika en samma-dags-feedback-loop mellan rekordhändelsen (som använder P_t) och bandbredden.
- **Exit-timing:** time stop definieras som exakt h handelsdagar efter entry, exekverad vid öppning (samma konvention som entry) — inte vid stängning.
- **Sizing:** k (skalär i $w_i = k/\hat{\theta}_i$) löses **iterativt**, inte i ett enda linjärt steg — ett enstegs-linjärt antagande visade sig ge upp till 50% volavvikelse när bruttotaket på 200% band (se commit-historik: detta var en verklig bugg, hittad och fixad under arbetets gång, se [tests/test_backtest.py::test_solve_k_for_target_vol_converges_when_gross_cap_binds](#)).
- **FIFO-tolkning:** en ny händelse som blockeras av fullt kapacitetstak (12 samtidiga) **förkastas**, köas inte till nästa lediga plats — ingen tidigare gren tillhandahöll ett könings-mönster att luta sig mot, och detta är den enklaste, mest standardmässiga tolkningen.
- **N_SUBPERIODS:** brief:ens "4 delperioder" tolkas som 4 lika stora kalenderperioder över hela 2004–2026-samplet (inte en fördefinierad, ojämn indelning).
- **Blockbootstrap:** cirkulär, fast blocklängd (20 dagar) — matchar Vindkastets [block_shuffle](#)-mönster snarare än Formdriftens [arch](#)-paket-baserade stationära bootstrap (funktionellt likvärdigt, hemodlat för att undvika ett extra beroende).

Köra om

```

pip install -r requirements.txt
export EODHD_API_KEY=... # data/*.csv är redan committade; endast nödvändigt vid --force-refetch
python -m research.dammluckan.run_calibration
python -m research.dammluckan.run_research
pytest research/dammluckan/tests/ -q

output/*.pkl (per-stadium-cache) är gitignored och regenereras automatiskt; output/
results_summary.json och output/grid_table.csv är de committade, läsbara resultatartefakterna som
REPORT.md citerar.

```

Resultatartefakter

Den fullständiga 27-cells griden och den sammanfattande resultat-JSON:en som rapportens siffror citerar.

27-cells grid (n × θ -percentil × h)

cell_id	n	theta_pctl	h	is_sharpe	oos1_sharpe	is_sharpe_pp	oos1_sharpe_pp	n_trades	n_trades_oos
n80_p70_h5	80	70	5	0.1789	0.1869	0.01127	0.01177	1303	738
n80_p70_h10	80	70	10	0.3529	0.2135	0.02223	0.01345	1153	651
n80_p70_h15	80	70	15	0.2055	0.2526	0.01294	0.01591	1077	612
n80_p80_h5	80	80	5	0.09764	0.0805	0.00615	0.005071	805	444
n80_p80_h10	80	80	10	0.4024	0.1704	0.02535	0.01073	735	404
n80_p80_h15	80	80	15	0.04185	0.3623	0.002637	0.02283	698	388
n80_p90_h5	80	90	5	0.3261	-0.01077	0.02054	-0.0006786	349	191
n80_p90_h10	80	90	10	0.3417	0.1568	0.02153	0.009878	326	180
n80_p90_h15	80	90	15	0.2966	0.4819	0.01868	0.03036	313	175
n120_p70_h5	120	70	5	0.157	0.3582	0.00989	0.02256	852	476
n120_p70_h10	120	70	10	0.552	0.5734	0.03477	0.03612	750	421
n120_p70_h15	120	70	15	0.2253	0.4924	0.01419	0.03102	704	398
n120_p80_h5	120	80	5	-0.03912	0.4315	-0.002465	0.02718	491	258
n120_p80_h10	120	80	10	0.3031	0.4205	0.01909	0.02649	443	234
n120_p80_h15	120	80	15	0.2131	0.2226	0.01342	0.01403	427	228
n120_p90_h5	120	90	5	-0.06837	0.4752	-0.004307	0.02994	179	95
n120_p90_h10	120	90	10	0.2021	0.6355	0.01273	0.04003	169	90
n120_p90_h15	120	90	15	-0.01859	0.6174	-0.001171	0.03889	167	89
n160_p70_h5	160	70	5	0.1417	0.7448	0.008926	0.04692	667	364
n160_p70_h10	160	70	10	0.1216	0.83	0.007657	0.05228	587	325
n160_p70_h15	160	70	15	-0.07791	0.2451	-0.004908	0.01544	553	312
n160_p80_h5	160	80	5	0.4813	0.6801	0.03032	0.04285	371	192
n160_p80_h10	160	80	10	0.4256	0.8432	0.02681	0.05312	338	175
n160_p80_h15	160	80	15	0.3364	0.3907	0.02119	0.02461	325	169
n160_p90_h5	160	90	5	0.3043	0.8429	0.01917	0.0531	124	65
n160_p90_h10	160	90	10	0.05674	0.7699	0.003574	0.0485	116	61
n160_p90_h15	160	90	15	0.0981	0.8166	0.00618	0.05144	114	60

results_summary.json

```
{
  "gate0": {
```

```

    "n_long": 479,
    "n_short": 254,
    "pass_count": false,
    "pass_concentration": true,
    "conc_long_max_share": 0.12108559498956159,
    "conc_long_max_ticker": "SPY",
    "conc_short_max_share": 0.10236220472440945,
    "conc_short_max_ticker": "SLV",
    "long_counts": {
      "SPY": 58,
      "IWM": 36,
      "EFA": 44,
      "EEM": 27,
      "TLT": 19,
      "IEF": 31,
      "LQD": 29,
      "HYG": 26,
      "GLD": 26,
      "SLV": 12,
      "DBC": 39,
      "USO": 42,
      "UUP": 24,
      "FXE": 19,
      "FXV": 26,
      "VNQ": 21
    },
    "short_counts": {
      "SPY": 15,
      "IWM": 15,
      "EFA": 9,
      "EEM": 11,
      "TLT": 24,
      "IEF": 14,
      "LQD": 22,
      "HYG": 8,
      "GLD": 8,
      "SLV": 26,
      "DBC": 12,
      "USO": 12,
      "UUP": 22,
      "FXE": 20,
      "FXV": 26,
      "VNQ": 10
    }
  },
  "gate1": {
    "ic": 0.06916094733651952,
    "n_events": 8842,
    "null_p_value": 0.005988023952095809,
    "null_n_draws": 500,
    "redundancy_r2": 0.005644890110424794,
    "redundancy_n_obs": 8611,
    "redundancy_spearman": {
      "vol": 0.030679394751598577,
      "skew": -0.011734108367944187,
      "autocorr_abs_r": -0.01650666971173899,
      "momentum_n": 0.10168192057499621,
      "dist_to_max": 0.14610963352362444,
      "tsmom_proxy": 0.08182422463996229
    },
    "pass_ic": true,
    "pass_redundancy": true
  },
  "primary_cell": {
    "n": 120,
    "theta_pctl": 80,
    "h": 10,
    "c": 1.30868720703125,
    "k": 0.0807071576555181
  },
  "primary": {
    "n_trades": 443,
    "k": 0.0807071576555181,
    "achieved_vol_is": 0.07999341032292939,
    "is_sharpe": 0.3031134634547036,
    "is_ann_return": 0.024247079656536368,
    "is_ann_vol": 0.07999341032292939,
    "oos1_sharpe": 0.4204664687009385,

```

```

    "oos1_ann_return": 0.039098838984462043,
    "oos1_ann_vol": 0.09298919627351196,
    "full_sharpe": 0.3473385467500183,
    "full_max_dd": -0.22338086301545512,
    "nw_tstat_full": 1.8894499261090183,
    "nw_tstat_oos1": 1.392791742295124
  },
  "donchian_twin": {
    "n_trades": 2430,
    "k": 0.01962095559295291,
    "achieved_vol_is": 0.08000101905602536,
    "is_sharpe": 0.2872830223763932,
    "oos1_sharpe": 0.44819519969954447,
    "full_sharpe": 0.3627749222043767,
    "full_max_dd": -0.144020459722754,
    "net_excess_is": 0.015830441078310387,
    "net_excess_oos1": -0.027728730998605955
  },
  "anti_twin": {
    "n_trades": 987,
    "k": 0.03164451731690563,
    "achieved_vol_is": 0.08000349258652623,
    "is_sharpe": -0.10651772187162695,
    "oos1_sharpe": -0.1683232954617636,
    "full_sharpe": -0.11315193429626766
  },
  "null3_block_permuted": {
    "n_draws": 500,
    "null_mean_pp_sharpe": -0.007453910605720798,
    "null_std_pp_sharpe": 0.01175171695810861,
    "real_pp_sharpe": 0.02188027179636025,
    "p_value": 0.00998003992015968
  },
  "null4_randomized_entry": {
    "n_draws": 500,
    "null_mean_pp_sharpe": 0.0008694671325306971,
    "null_std_pp_sharpe": 0.012217021340151794,
    "p_value": 0.0499001996007984
  },
  "tsmom_corr_full": 0.049626653476029944,
  "tsmom_corr_oos1": 0.009022398801015397,
  "sign_consistency": {
    "full_sign": 1.0,
    "period_signs": [
      1.0,
      -1.0,
      1.0,
      1.0
    ],
    "period_ann_returns": [
      0.06715920568421234,
      -0.012476425247484364,
      0.023729890920496086,
      0.0404220004929637
    ],
    "n_inconsistent": 1,
    "boundaries": [
      [
        "2004-01-01T00:00:00",
        "2009-08-25T12:00:00"
      ],
      [
        "2009-08-25T12:00:00",
        "2015-04-20T00:00:00"
      ],
      [
        "2015-04-20T00:00:00",
        "2020-12-12T12:00:00"
      ],
      [
        "2020-12-12T12:00:00",
        "2026-08-07T00:00:00"
      ]
    ]
  },
  "pnl_quarter_concentration": 0.04242938508912997,
  "dsr_is": {
    "dsr_prob": 0.44114241938413273,

```

```

    "dsr_excess": -0.0023954705660142843,
    "expected_max_sr": 0.021489823978790734
  },
  "dsr_oos1": {
    "dsr_prob": 0.3841983318440866,
    "dsr_excess": -0.006224769865830041,
    "expected_max_sr": 0.032711667742596866
  },
  "neighborhood_is": {
    "target_sign": 1.0,
    "n_neighbors": 6,
    "n_same_sign": 5,
    "frac_same_sign": 0.8333333333333334,
    "passes": "True"
  },
  "neighborhood_oos1": {
    "target_sign": 1.0,
    "n_neighbors": 6,
    "n_same_sign": 6,
    "frac_same_sign": 1.0,
    "passes": "True"
  },
  "primary_cell_id": "n120_p80_h10",
  "oos2": {
    "n_long": 683,
    "n_short": 226,
    "conc_long": {
      "n_events": 683,
      "counts": {
        "EWJ": 44,
        "EWG": 33,
        "EWU": 43,
        "EWQ": 42,
        "EWI": 44,
        "EWP": 27,
        "EWL": 49,
        "EWA": 62,
        "EWC": 60,
        "EWY": 33,
        "EWT": 39,
        "EWZ": 38,
        "EWW": 47,
        "EWS": 37,
        "EWH": 27,
        "EWD": 58
      }
    },
    "max_share": 0.09077598828696926,
    "max_ticker": "EWA"
  },
  "conc_short": {
    "n_events": 226,
    "counts": {
      "EWJ": 20,
      "EWG": 8,
      "EWU": 10,
      "EWQ": 10,
      "EWI": 14,
      "EWP": 8,
      "EWL": 8,
      "EWA": 8,
      "EWC": 11,
      "EWY": 21,
      "EWT": 13,
      "EWZ": 12,
      "EWW": 33,
      "EWS": 14,
      "EWH": 13,
      "EWD": 23
    }
  },
  "max_share": 0.14601769911504425,
  "max_ticker": "EWW"
},
"n_trades": 514,
"sharpe_full": 0.25206587522234203,
"ann_return": 0.02727419810997703,
"ann_vol": 0.10820265966553001,
"donchian_sharpe": 0.08363877295770757

```

Del II — Källkod

Samtliga 20 Python-filer (2405 rader) i läsordning: konfiguration och data först, därefter signal, motor, statistiskt maskineri, orkestrering och tester.

```

"""
Daml Luckan -- shared configuration and declared parameters.

Every constant that is DECLARED (i.e. not literally pinned by the brief) is
called out as such in a comment, following the house convention used by the
sibling research branches (Vridmomentet, Vindkastet, Formdriften): nothing
here is silently assumed.
"""

from dataclasses import dataclass, field

# -----
# Universe (reused verbatim from research/vindkastet/fetch_data.py)
# -----
PRIMARY = ["SPY", "IWM", "EFA", "EEM", "TLT", "IEF", "LQD", "HYG", "GLD",
           "SLV", "DBC", "USO", "UUP", "FXE", "FXV", "VNQ"]
SECONDARY = ["EWJ", "EWG", "EWU", "EWQ", "EWI", "EWP", "EWL", "EWA", "EWC",
             "EWY", "EWT", "EWZ", "EWW", "EWS", "EWH", "EWD"]

# -----
# Sample split (IS/OOS-1 boundary pinned by the brief; OOS-2 is the secondary
# universe, touched exactly once, at the very end of the protocol).
# -----
IS_START = "2004-01-01"
IS_END = "2017-12-31"
OOS1_START = "2018-01-01"
OOS1_END = "2026-08-07"
HISTORY_START = "2003-01-01"

# last EODHD bar actually fetched
# DECLARED: warm-up buffer before IS_START so
# the n=160 rolling window has a full look-
# back available on the very first IS date.

# -----
# Signal parameters (primary cell; grid varies n, theta_ctl, h around this)
# -----
N_DEFAULT = 120
THETA_PCTL_DEFAULT = 80
H_DEFAULT = 10

N_GRID = (80, 120, 160)
THETA_PCTL_GRID = (70, 80, 90)
H_GRID = (5, 10, 15)

# DECLARED: sigma-hat used only to normalize the occupation-band width is a
# trailing realized-vol estimate of daily log returns (raw close), matching
# the vol60 convention used throughout the sibling branches.
VOL_LOOKBACK = 60
VOL_MIN_PERIODS = 45

# DECLARED: band-width constant c is calibrated (once, on the primary
# universe, IS window only) so that the block-bootstrap null median of 0+
# (and, by symmetry of the null construction, 0-) is approx. 0.15, per the
# brief's calibration instruction. See signal.calibrate_band_constant().
# The frozen value is filled in by calibrate_band_constant() and cached to
# output/band_constant.json so OOS runs reuse the exact IS-frozen value.
C_BAND_TARGET_MEDIAN = 0.15
BAND_HORIZON_DAYS = 5

# the "sqrt(5)" horizon scaling in the brief

# -----
# Block-bootstrap null (used for theta_i calibration, c calibration, Steg-1
# IC null test, and the portfolio-level null-twin #3).
# -----
BLOCK_LENGTH = 20

# DECLARED: ~1 trading month, matches the
# block length formdriften's nulls.py used
# for its own estimator null.

N_BLOCK_DRAWS = 500
N_THETA_CALIB_DRAWS = 1000

# DECLARED: null-distribution resamples
# DECLARED: per-asset draws for theta_i / c

# -----
# Portfolio / sizing
# -----
MAX_CONCURRENT = 12
GROSS_CAP = 2.0

# 200%
PORTFOLIO_VOL_TARGET = 0.08
# 8% annualized ex-ante
SIZING_VOL_LOOKBACK = 20

# DECLARED: daily realized-vol lookback used
# for sigma_hat_i in w_i = vol_target/sigma_hat_i

TRADING_DAYS_YEAR = 252

# -----
# Costs -- ADV-bucket model, ported verbatim from research/formdriften/costs.py

```

```

# (the only ADV-bucketed cost model anywhere in the research-branch series;
# "reports ADV-bucketmodell" in the brief refers to this one).
# -----
COMMISSION_BPS = 3.0
ADV_BUCKETS = ( # (min ADV $, half-spread bps), most liquid first
    (500e6, 0.5),
    (200e6, 1.0),
    (100e6, 1.5),
    (50e6, 2.5),
    (20e6, 4.0),
    (0.0, 7.0),
)
ADV_COST_LOOKBACK = 63
# DOCUMENTED SPREAD CAVEAT (verbatim convention from Formdriften): EODHD does
# not provide quoted bid/ask spread history, so the half-spread leg is
# approximated from trailing dollar-ADV via a monotone liquidity bucket, not
# measured from actual NBBO data. Only the commission leg (3bp/side) is a
# clean, un-approximated assumption.

# -----
# Steg 0 / Steg 1 gates
# -----
MIN_EVENTS_PER_SIDE = 300
MAX_SINGLE_ASSET_EVENT_SHARE = 0.5
IC_P_KILL = 0.05
IC_ABS_KILL = 0.03

# -----
# Kill criteria (pre-registered, frozen before 00S-1/00S-2 are inspected)
# -----
DSR_OOS_KILL = 0.0
TSMOM_CORR_KILL = 0.4
SIGN_INCONSISTENCY_SUBPERIODS_KILL = 2 # of 4
NET_EXCESS_OVER_DONCHIAN_TWIN_KILL = 0.0

N_SUBPERIODS = 4 # brief: "teckeninkonsistens i >= 2 av 4 delperioder" --
# DECLARED: the full IS+00S-1 sample (2004-2026) is cut
# into 4 equal-length calendar sub-periods; see
# stats.sub_period_boundaries().

@dataclass(frozen=True)
class SignalParams:
    n: int = N_DEFAULT
    theta_pctl: float = THETA_PCTL_DEFAULT
    h: int = H_DEFAULT
    c_band: float = None # filled in after calibration

@dataclass(frozen=True)
class PortfolioParams:
    max_concurrent: int = MAX_CONCURRENT
    gross_cap: float = GROSS_CAP
    vol_target: float = PORTFOLIO_VOL_TARGET
    sizing_vol_lookback: int = SIZING_VOL_LOOKBACK

```

```

"""
Dammluckan -- data fetcher.

Pulls daily OHLCV (raw + adjusted close) from EODHD for:
- the 16-ETF primary multi-asset universe (equity idx, duration, credit,
  gold, oil, broad commodity, USD-proxy, EM) -- the same panel used by the
  Vindkastet research branch (research/vindkastet/fetch_data.py), reused
  here verbatim as "the existing EODHD panel" the brief refers to.
- the 16 single-country equity ETFs reserved as the untouched OOS-2
  confirmation surface (identical list to Vindkastet's secondary universe).

Caches raw JSON responses to research/dammluckan/data/raw/ and writes a
combined, NaN-preserving (no forward-fill) OHLCV panel per universe to
research/dammluckan/data/{primary,secondary}<field>.csv.
"""
import os
import json
import time
import urllib.request
import urllib.error
import ssl
import certifi

API_KEY = os.environ["EODHD_API_KEY"]
BASE = "https://eodhd.com/api/eod/{sym}.US"
FROM = "2003-01-01"
TO = "2026-08-10"

HERE = os.path.dirname(os.path.abspath(__file__))
RAW_DIR = os.path.join(HERE, "data", "raw")
os.makedirs(RAW_DIR, exist_ok=True)

# Primary 16-ETF multi-asset universe (reused verbatim from Vindkastet):
# equity idx (SPY, IWM, EFA, EEM), duration (TLT, IEF), credit (LQD, HYG),
# gold (GLD), silver (SLV), broad commodity (DBC), oil (USO), USD-proxy (UUP),
# FX (FXE, FXV), REIT (VNQ).
PRIMARY = ["SPY", "IWM", "EFA", "EEM", "TLT", "IEF", "LQD", "HYG", "GLD",
           "SLV", "DBC", "USO", "UUP", "FXE", "FXV", "VNQ"]

# Reserved confirmation surface: 16 single-country equity ETFs, touched
# exactly once, last (OOS-2). Identical list to Vindkastet's secondary
# universe -- never used for calibration/tuning of this hypothesis.
SECONDARY = ["EWJ", "EWG", "EWU", "EWQ", "EWI", "EWP", "EWL", "EWA", "EWC",
             "EWY", "EWT", "EWZ", "EWW", "EWS", "EWH", "EWD"]

FIELDS = ["open", "high", "low", "close", "adjusted_close", "volume"]

CTX = ssl.create_default_context(cafile=os.environ.get("SSL_CERT_FILE", certifi.where()))

def _fetch_one(sym, retries=4):
    cache_path = os.path.join(RAW_DIR, f"{sym}.json")
    if os.path.exists(cache_path):
        with open(cache_path) as f:
            return json.load(f)
    url = BASE.format(sym=sym) + f"?api_token={API_KEY}&period=d&fmt=json&from={FROM}&to={TO}"
    proxy = os.environ.get("HTTPS_PROXY") or os.environ.get("https_proxy")
    handlers = []
    if proxy:
        handlers.append(urllib.request.ProxyHandler({"https": proxy, "http": proxy}))
    handlers.append(urllib.request.HTTPSHandler(context=CTX))
    opener = urllib.request.build_opener(*handlers)
    last_err = None
    for attempt in range(retries):
        try:
            with opener.open(url, timeout=30) as resp:
                data = json.loads(resp.read().decode())
            with open(cache_path, "w") as f:
                json.dump(data, f)
            return data
        except Exception as e:
            last_err = e
            time.sleep(1.5 * (attempt + 1))
    raise RuntimeError(f"Failed to fetch {sym}: {last_err}")

def fetch_all(symbols):
    out = {}

```

```

for sym in symbols:
    data = _fetch_one(sym)
    if isinstance(data, dict) and "code" in data and "message" in data and len(data) <= 3:
        print(f"WARNING: {sym} returned error payload: {data}")
        continue
    out[sym] = data
    print(f"{sym}: {len(data)} rows, {data[0]['date']} if data else 'EMPTY' -> {data[-1]['date']} if data else
'EMPTY'")
    return out

if __name__ == "__main__":
    import pandas as pd

    for label, syms in [("primary", PRIMARY), ("secondary", SECONDARY)]:
        print(f"\n=== Fetching {label} universe ===")
        raw = fetch_all(syms)
        for field in FIELDS:
            panel = {}
            for sym, rows in raw.items():
                s = pd.Series({r["date"]: r.get(field) for r in rows if r.get(field) is not None})
                s.index = pd.to_datetime(s.index)
                panel[sym] = s
            df = pd.DataFrame(panel).sort_index()
            out_path = os.path.join(HERE, "data", f"{label}_{field}.csv")
            df.to_csv(out_path)
        print(f"Saved {label}: shape={df.shape}, date range {df.index.min()} -> {df.index.max()}")
        print("Live-name count (last row):", df.iloc[-1].notna().sum())
        print("First valid index per column:\n", df.apply(lambda c: c.first_valid_index()))

```

```

"""
Dammluckan -- panel construction from the cached EODHD CSVs.

Design choice (DECLARED, and worth stating explicitly): record/occupation
detection (M_t, E+/E-, O+/O-) runs on the RAW (unadjusted) close, because the
hypothesis is about anchoring to actual historical traded price levels
(resting limit orders parked at literal past highs/lows) -- using a
dividend-adjusted series would silently shift those historical levels for
higher-yielding names (HYG, LQD, TLT, VNQ, EEM...) and distort exactly the
mechanism being tested. Everything downstream of the entry decision (returns,
volatility, sizing, Sharpe) uses the adjusted series, which is the standard,
correct convention for total-return performance measurement.
"""

import os
import numpy as np
import pandas as pd
from dataclasses import dataclass, field

from . import config

HERE = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(HERE, "data")

FIELDS = ["open", "high", "low", "close", "adjusted_close", "volume"]

def _load_field(universe_label, field):
    path = os.path.join(DATA_DIR, f"{universe_label}_{field}.csv")
    df = pd.read_csv(path, index_col=0, parse_dates=True)
    return df.sort_index()

@dataclass
class Panel:
    tickers: list
    raw_close: pd.DataFrame
    raw_open: pd.DataFrame
    high: pd.DataFrame
    low: pd.DataFrame
    adj_close: pd.DataFrame
    adj_open: pd.DataFrame
    volume: pd.DataFrame
    dollar_volume: pd.DataFrame = field(init=False)
    log_returns: pd.DataFrame = field(init=False)
    band_vol: pd.DataFrame = field(init=False) # sigma_hat for O+/O- band width
    sizing_vol: pd.DataFrame = field(init=False) # sigma_hat for position sizing
    adv: pd.DataFrame = field(init=False) # trailing dollar ADV for costs

    def __post_init__(self):
        # dollar volume uses RAW close (matches Vridmomentet convention:
        # "close * volume (raw close, not adjusted)")
        self.dollar_volume = self.raw_close * self.volume
        self.log_returns = np.log(self.adj_close / self.adj_close.shift(1))
        self.band_vol = self.log_returns.rolling(
            config.VOL_LOOKBACK, min_periods=config.VOL_MIN_PERIODS
        ).std(ddof=1)
        self.sizing_vol = self.log_returns.rolling(
            config.SIZING_VOL_LOOKBACK, min_periods=config.SIZING_VOL_LOOKBACK
        ).std(ddof=1)
        self.adv = self.dollar_volume.rolling(
            config.ADV_COST_LOOKBACK, min_periods=config.ADV_COST_LOOKBACK // 2
        ).mean()

    @property
    def dates(self):
        return self.raw_close.index

    def eligible_on(self, as_of) -> pd.Index:
        """PIT-eligible: ticker has traded (non-NaN raw close) as of this date."""
        row = self.raw_close.loc[as_of]
        return row[row.notna()].index

    def slice(self, start=None, end=None) -> "Panel":
        def cut(df):
            return df.loc[start:end]
        return Panel(
            tickers=self.tickers,
            raw_close=cut(self.raw_close), raw_open=cut(self.raw_open),

```

```

        high=cut(self.high), low=cut(self.low),
        adj_close=cut(self.adj_close), adj_open=cut(self.adj_open),
        volume=cut(self.volume),
    )

def _adjust_open(raw_open, raw_close, adj_close):
    """Reconstruct adjusted open from the adj_close/close cumulative factor.

    Same mechanic as Vridmomentet's _adjust_ohlcv: EODHD only gives an
    adjusted close, so open/high/low are backed out via the ratio.
    """
    factor = adj_close / raw_close
    return raw_open * factor

def build_panel(universe_label) -> Panel:
    raw_open = _load_field(universe_label, "open")
    raw_close = _load_field(universe_label, "close")
    high = _load_field(universe_label, "high")
    low = _load_field(universe_label, "low")
    adj_close = _load_field(universe_label, "adjusted_close")
    volume = _load_field(universe_label, "volume")

    # Widen all frames to a shared union index/columns without forward-fill:
    # absence of data means absence of eligibility, not a stale carried price.
    idx = raw_close.index
    cols = raw_close.columns
    raw_open = raw_open.reindex(index=idx, columns=cols)
    high = high.reindex(index=idx, columns=cols)
    low = low.reindex(index=idx, columns=cols)
    adj_close = adj_close.reindex(index=idx, columns=cols)
    volume = volume.reindex(index=idx, columns=cols)

    adj_open = _adjust_open(raw_open, raw_close, adj_close)

    tickers = list(cols)
    return Panel(
        tickers=tickers,
        raw_close=raw_close, raw_open=raw_open,
        high=high, low=low,
        adj_close=adj_close, adj_open=adj_open,
        volume=volume,
    )

def load_primary_panel() -> Panel:
    return build_panel("primary")

def load_secondary_panel() -> Panel:
    return build_panel("secondary")

```



```

"""
Dammluckan -- record-hazard signal.

M_t = rolling max/min of RAW close over [t-n, t-1] (causal: excludes t)
E+_t = 1{P_t > M_t} (upside record)
E-_t = 1{P_t < m_t} (downside record)
O+_t = share of days in [t-n, t-1] with P_s >= M_t * (1 - c * band_vol_t * sqrt(5))
O-_t = share of days in [t-n, t-1] with P_s <= m_t * (1 + c * band_vol_t * sqrt(5))

band_vol_t is the asset's own trailing realized vol (config.VOL_LOOKBACK,
daily log returns), evaluated strictly through t-1 (shifted by one day
relative to data.Panel.band_vol) so the occupation band at decision time t
never uses day t's own return -- avoiding a same-day feedback loop between
the record event (which does use P_t) and the band width.

c is calibrated once (per grid window length n) on the primary universe's
IS-only block-bootstrap null so that the pooled null median of O+ (and, by
the mirrored construction, O-) is approximately 0.15 -- see
calibrate_band_constant(). theta_i is calibrated per-asset, per-n, as the
80th (grid: 70/80/90th) percentile of that asset's own block-bootstrap null
distribution of O+ -- see calibrate_theta().
"""
import warnings

import numpy as np
import pandas as pd
from numpy.lib.stride_tricks import sliding_window_view

from . import config
from . import nulls

def rolling_max_excl_today(price: pd.DataFrame, n: int) -> pd.DataFrame:
    """M_t = max(P_s for s in [t-n, t-1])."""
    return price.shift(1).rolling(n, min_periods=n).max()

def rolling_min_excl_today(price: pd.DataFrame, n: int) -> pd.DataFrame:
    return price.shift(1).rolling(n, min_periods=n).min()

def record_events_high(price: pd.DataFrame, m_t: pd.DataFrame) -> pd.DataFrame:
    valid = m_t.notna() & price.notna()
    return (price > m_t).astype(float).where(valid)

def record_events_low(price: pd.DataFrame, m_t: pd.DataFrame) -> pd.DataFrame:
    valid = m_t.notna() & price.notna()
    return (price < m_t).astype(float).where(valid)

def _occupation_high_ld(prices: np.ndarray, band_vol_causal: np.ndarray, n: int, c: float) -> np.ndarray:
    """O+_t for one ticker's raw close series (see module docstring)."""
    T = len(prices)
    out = np.full(T, np.nan)
    if T <= n:
        return out
    shifted = prices[:-1]
    windows = sliding_window_view(shifted, n) # windows[k] = prices[k:k+n], k=0..T-n-1 -> t=k+n
    valid_count = np.sum(~np.isnan(windows), axis=1)
    with np.errstate(invalid="ignore"), warnings.catch_warnings():
        warnings.filterwarnings("ignore", message="All-NaN slice encountered")
        m_t = np.nanmax(windows, axis=1)
        band_vol_t = band_vol_causal[n:]
        thresh = m_t * (1.0 - c * band_vol_t * np.sqrt(config.BAND_HORIZON_DAYS))
        occ = np.sum(windows >= thresh[:, None], axis=1) / n
        occ = np.where((valid_count == n) & np.isfinite(thresh), occ, np.nan)
    out[n:] = occ
    return out

def _occupation_low_ld(prices: np.ndarray, band_vol_causal: np.ndarray, n: int, c: float) -> np.ndarray:
    T = len(prices)
    out = np.full(T, np.nan)
    if T <= n:
        return out
    shifted = prices[:-1]
    windows = sliding_window_view(shifted, n)

```

```

valid_count = np.sum(~np.isnan(windows), axis=1)
with np.errstate(invalid="ignore"), warnings.catch_warnings():
    warnings.filterwarnings("ignore", message="All-NaN slice encountered")
    m_t = np.nanmin(windows, axis=1)
band_vol_t = band_vol_causal[n:]
thresh = m_t * (1.0 + c * band_vol_t * np.sqrt(config.BAND_HORIZON_DAYS))
occ = np.sum(windows <= thresh[:, None], axis=1) / n
occ = np.where((valid_count == n) & np.isfinite(thresh), occ, np.nan)
out[n:] = occ
return out

def occupation_high(price: pd.DataFrame, band_vol_causal: pd.DataFrame, n: int, c: float) -> pd.DataFrame:
    out = {}
    for col in price.columns:
        out[col] = _occupation_high_1d(price[col].to_numpy(), band_vol_causal[col].to_numpy(), n, c)
    return pd.DataFrame(out, index=price.index)

def occupation_low(price: pd.DataFrame, band_vol_causal: pd.DataFrame, n: int, c: float) -> pd.DataFrame:
    out = {}
    for col in price.columns:
        out[col] = _occupation_low_1d(price[col].to_numpy(), band_vol_causal[col].to_numpy(), n, c)
    return pd.DataFrame(out, index=price.index)

class SignalBundle:
    """All signal artifacts for one (n, c) configuration, for a given panel."""

    __slots__ = ("n", "c", "m_high", "m_low", "e_high", "e_low", "o_high", "o_low", "band_vol_causal")

    def __init__(self, panel, n: int, c: float):
        self.n = n
        self.c = c
        self.band_vol_causal = panel.band_vol.shift(1)
        self.m_high = rolling_max_excl_today(panel.raw_close, n)
        self.m_low = rolling_min_excl_today(panel.raw_close, n)
        self.e_high = record_events_high(panel.raw_close, self.m_high)
        self.e_low = record_events_low(panel.raw_close, self.m_low)
        self.o_high = occupation_high(panel.raw_close, self.band_vol_causal, n, c)
        self.o_low = occupation_low(panel.raw_close, self.band_vol_causal, n, c)

def build_signal(panel, n: int, c: float) -> SignalBundle:
    return SignalBundle(panel, n, c)

# -----
# Calibration: band constant c (per n) and per-asset theta_i (per n, per pctl)
# -----

def calibrate_band_constant(panel, n: int, is_start=config.IS_START, is_end=config.IS_END,
                           target_median=config.C_BAND_TARGET_MEDIAN,
                           n_draws=200, seed=0) -> float:
    """
    Bisection search for c such that the pooled (across all primary-universe
    assets, across all bootstrap draws) null median of 0+ over the IS window
    is approximately target_median. IS-only, frozen for OOS use.
    """
    is_panel = panel.slice(is_start, is_end)
    lo, hi = 1e-4, 20.0

    def median_occ_for_c(c):
        pooled = nulls.pooled_null_occupation(is_panel, n=n, c=c, side="high", n_draws=n_draws, seed=seed)
        return np.nanmedian(pooled)

    med_lo, med_hi = median_occ_for_c(lo), median_occ_for_c(hi)
    for _ in range(24):
        mid = 0.5 * (lo + hi)
        med_mid = median_occ_for_c(mid)
        if med_mid > target_median:
            hi, med_hi = mid, med_mid
        else:
            lo, med_lo = mid, med_mid
        if abs(med_mid - target_median) < 1e-4:
            break
    return 0.5 * (lo + hi)

def calibrate_theta_multi(panel, n: int, c: float, pctls, side="high",
                          is_start=config.IS_START, is_end=config.IS_END,
                          n_draws=config.N_THETA_CALIB_DRAWS, seed=0) -> dict:

```

```

"""
theta_i = pct1-th percentile of asset i's own block-bootstrap null
distribution of 0+ (side='high') / 0- (side='low'), estimated IS-only,
frozen for OOS use. Generates the null ONCE per asset and reads off every
requested percentile from it (avoids redundant bootstrap draws across the
theta_pct1 grid).

Returns {pctl: pd.Series indexed by ticker}.
"""
is_panel = panel.slice(is_start, is_end)
per_pct1 = {p: {} for p in pctls}
for i, ticker in enumerate(panel.tickers):
    null_vals = nulls.per_asset_null_occupation(
        is_panel, ticker=ticker, n=n, c=c, side=side, n_draws=n_draws, seed=seed + i
    )
    for p in pctls:
        per_pct1[p][ticker] = np.nanpercentile(null_vals, p) if len(null_vals) else np.nan
return {p: pd.Series(vals) for p, vals in per_pct1.items()}

```

```

"""
Daml Luckan -- block-bootstrap null primitives.

A single circular block-bootstrap resampler of daily log returns, used for:
(a) calibrating the band constant c (signal.calibrate_band_constant)
(b) calibrating per-asset theta_i (signal.calibrate_theta)
(c) the Steg-1 estimator-level IC null test (robustness.ic_null_test)
(d) null-twin #3, "block-permuted returns per asset -> the full rule set on
    synthetic paths" (battery.block_permuted_returns_null)

Block length is config.BLOCK_LENGTH (~1 trading month), matching the
convention Formdriften's own estimator null used for the same purpose.
"""
import numpy as np
import pandas as pd

from . import config

def block_bootstrap_indices(n_obs: int, length: int, block: int, rng: np.random.Generator) -> np.ndarray:
    """Circular block-bootstrap index array of len `length`, drawn from a
    series of length n_obs (wraps around so every start point is usable)."""
    n_blocks = int(np.ceil(length / block))
    starts = rng.integers(0, n_obs, size=n_blocks)
    idx = np.concatenate([(np.arange(s, s + block) % n_obs) for s in starts][:length])
    return idx

def block_bootstrap_returns(returns: np.ndarray, length: int, rng: np.random.Generator,
                           block: int = config.BLOCK_LENGTH) -> np.ndarray:
    """Circular block-bootstrap resample of a 1D return array to `length`."""
    idx = block_bootstrap_indices(len(returns), length, block, rng)
    return returns[idx]

def synthetic_price_path(returns: np.ndarray, length: int, rng: np.random.Generator,
                        start_price: float = 100.0, block: int = config.BLOCK_LENGTH) -> np.ndarray:
    """Reconstruct a synthetic raw-close path from a block-bootstrapped log-return draw."""
    r = block_bootstrap_returns(returns, length, rng, block=block)
    log_path = np.log(start_price) + np.concatenate([[0.0], np.cumsum(r)])
    return np.exp(log_path)

def _asset_log_returns(panel, ticker) -> np.ndarray:
    s = panel.raw_close[ticker].dropna()
    r = np.log(s / s.shift(1)).dropna().to_numpy()
    return r

def per_asset_null_occupation(panel, ticker: str, n: int, c: float, side: str,
                             n_draws: int, seed: int, path_length: int = None) -> np.ndarray:
    """
    Pooled null distribution of 0+ (side='high') or 0- (side='low') for one
    asset: n_draws synthetic block-bootstrap price paths of the asset's own
    IS return history, each scored with the real (n, c) occupation formula,
    pooled (post warm-up) into one array.
    """
    from . import signal # local import: signal.py imports nulls.py

    r = _asset_log_returns(panel, ticker)
    if len(r) <= n + config.VOL_LOOKBACK:
        return np.array([])
    length = path_length or len(r) + 1
    rng = np.random.default_rng(seed)
    pooled = []
    occ_fn = signal.occupation_high_1d if side == "high" else signal.occupation_low_1d
    for draw in range(n_draws):
        path = synthetic_price_path(r, length - 1, rng, block=config.BLOCK_LENGTH)
        log_r = np.diff(np.log(path))
        band_vol = pd.Series(log_r).rolling(
            config.VOL_LOOKBACK, min_periods=config.VOL_MIN_PERIODS
        ).std(ddof=1).shift(1).to_numpy()
        band_vol = np.concatenate([[np.nan], band_vol]) # align to price path length
        occ = occ_fn(path, band_vol, n, c)
        pooled.append(occ[~np.isnan(occ)])
    return np.concatenate(pooled) if pooled else np.array([])

```

```

def pooled_null_occupation(panel, n: int, c: float, side: str, n_draws: int, seed: int) -> np.ndarray:
    """Same as per_asset_null_occupation but pooled across every ticker in
    the panel (used to calibrate the shared band constant c)."""
    pooled = []
    for i, ticker in enumerate(panel.tickers):
        vals = per_asset_null_occupation(panel, ticker, n, c, side, n_draws, seed=seed + 7919 * i)
        pooled.append(vals)
    return np.concatenate(pooled) if pooled else np.array([])

```

```

"""
Dammluckan -- ADV-bucket transaction cost model.

Ported verbatim (bucket thresholds, bps schedule, and the documented spread
caveat) from research/formdriften/costs.py -- the only ADV-bucketed cost
model anywhere in this repo's research-branch series. This is what the brief
means by "Kostnader ur repots ADV-bucketmodell (med dess dokumenterade
spread-caveat)".

DOCUMENTED SPREAD CAVEAT (verbatim convention, restated here): EODHD does not
provide quoted bid/ask spread history, so the half-spread leg below is
approximated from trailing dollar-ADV via a monotone liquidity bucket --
a documented approximation, not a measured spread. Only the commission leg
(3bp/side) is a clean, un-approximated assumption.
"""

import numpy as np
import pandas as pd

from . import config

_ADV_BUCKETS = config.ADV_BUCKETS # (min ADV $, half-spread bps), most liquid first


def half_spread_bps(adv_dollars):
    """Vectorized monotone ADV-bucket lookup. Accepts scalar, ndarray, or Series.

    Buckets are applied most-liquid-first: the first (highest) threshold an
    ADV value clears wins, so NaN ADV (not-yet-eligible-for-ADV history)
    falls through to the illiquid default bucket rather than raising.
    """
    arr = np.asarray(adv_dollars, dtype=float)
    out = np.full(arr.shape, _ADV_BUCKETS[-1][1], dtype=float)
    # Apply ascending by threshold so higher (more-liquid) thresholds are
    # applied last and correctly overwrite the looser bucket.
    for min_adv, bps in sorted(_ADV_BUCKETS, key=lambda x: x[0]):
        out = np.where(arr >= min_adv, bps, out)
    if np.isscalar(adv_dollars):
        return float(out)
    if isinstance(adv_dollars, pd.Series):
        return pd.Series(out, index=adv_dollars.index)
    return out


def round_trip_cost_bps(adv_dollars):
    return config.COMMISSION_BPS + half_spread_bps(adv_dollars)


def cost_fraction(adv_dollars):
    """Round-trip cost as a fraction of notional (not bps)."""
    return round_trip_cost_bps(adv_dollars) / 10_000.0

```

```

"""
Dammluckan -- position/trade primitives and portfolio-level sizing.

No sibling research branch provides genuine overlapping multi-day position
tracking (each of them is a single-book, full-replacement weekly/monthly
rebalance) -- this module and backtest.py implement that machinery fresh,
as an explicit FIFO-capped, event-driven position ledger.
"""

from dataclasses import dataclass
import numpy as np
import pandas as pd

from . import config
from . import costs

@dataclass
class CandidateTrade:
    """An asset-local candidate trade: what WOULD happen if this trigger
    were admitted, computed independent of portfolio-level constraints."""
    ticker: str
    direction: int          # +1 long, -1 short
    decision_date: pd.Timestamp
    entry_date: pd.Timestamp
    entry_pos: int          # integer position in the panel's calendar index
    exit_date: pd.Timestamp
    exit_pos: int
    exit_reason: str        # "time_stop" | "opposite_record"
    entry_price: float
    exit_price: float
    o_value: float          # occupation at decision (tie-break priority)
    entry_adv: float
    exit_adv: float

@dataclass
class Trade(CandidateTrade):
    weight: float = 0.0      # signed target weight, post gross-cap haircut
    sigma_hat: float = np.nan
    gross_return: float = 0.0 # direction * (exit/entry - 1)
    cost_frac: float = 0.0    # round-trip cost fraction (of notional)
    net_pnl_contribution: float = 0.0 # weight * gross_return - |weight| * cost_frac

def inverse_vol_weight(sigma_hat: float, k: float) -> float:
    """w_i magnitude = k / sigma_hat_i (k plays the role of sigma_target,
    solved for by backtest._solve_k_for_target_vol so ex-ante portfolio vol
    ~ config.PORTFOLIO_VOL_TARGET)."""
    if sigma_hat is None or not np.isfinite(sigma_hat) or sigma_hat <= 0:
        return 0.0
    return k / sigma_hat

def trade_cost_fraction(entry_adv: float, exit_adv: float) -> float:
    """Round-trip ADV-bucket cost, split half at entry (measured at entry
    ADV) and half at exit (measured at exit ADV) -- see costs.py."""
    entry_leg = costs.cost_fraction(entry_adv) / 2.0 if np.isfinite(entry_adv) else costs.cost_fraction(0.0) / 2.0
    exit_leg = costs.cost_fraction(exit_adv) / 2.0 if np.isfinite(exit_adv) else costs.cost_fraction(0.0) / 2.0
    return entry_leg + exit_leg

```

```

"""
Daml Luckan -- event-driven backtest engine.

Pipeline: generate_candidates() -> admit_candidates() (FIFO/gross-cap ledger)
-> assign_weights() (inverse-vol sizing + gross-cap haircut, event-driven,
no daily re-sizing) -> daily_returns() (exact open/close leg accounting).

Entry: decided at close t (E+/E- fires and 0+/0- >= theta_i), executed at
t+1 open. Exit: h-day time stop or the first opposite-direction record,
whichever executes first; a new same-direction event while a position is
open is ignored (no pyramiding, no re-entry, per the brief). FIFO cap on
concurrent positions: an asset already holding a position ignores new
triggers; when the book is at MAX_CONCURRENT, a new trigger for a flat asset
is simply forgone (not queued) -- this is itself part of the hypothesis's
"episodic, capacity-limited" character, not a modeling shortcut.
"""
import heapq
import numpy as np
import pandas as pd

from . import config
from . import costs
from . import metrics
from .portfolio import CandidateTrade, Trade, inverse_vol_weight, trade_cost_fraction

def compute_exit(entry_pos, h, opp_events, T):
    """Given an entry position, the h-day time-stop horizon, and the
    opposite-direction event array, return (exit_pos, exit_reason) --
    whichever of time-stop / first-opposite-record executes first. Shared by
    generate_candidates() and the randomized-entry null (battery.py)."""
    exit_pos_timestop = min(entry_pos + h, T - 1)
    scan_end = min(entry_pos + h - 1, T)
    exit_reason, exit_pos = "time_stop", exit_pos_timestop
    if scan_end > entry_pos:
        window = opp_events[entry_pos:scan_end]
        hits = np.flatnonzero(window == 1)
        if len(hits):
            candidate_exit = entry_pos + hits[0] + 1
            if candidate_exit < exit_pos_timestop:
                exit_pos, exit_reason = candidate_exit, "opposite_record"
    return exit_pos, exit_reason

def generate_candidates(panel, sig, theta_high, theta_low, h, comparator="ge"):
    """comparator='ge': occupation-gated primary/Donchian-twin family
    (0 >= theta, theta=-inf for the ungated Donchian twin). comparator='le':
    anti-twin family (0 <= theta, a LOW-occupation threshold)."""
    calendar = panel.dates
    T = len(calendar)
    candidates = []
    cmp = (lambda o, th: o >= th) if comparator == "ge" else (lambda o, th: o <= th)

    for ticker in panel.tickers:
        e_high = sig.e_high[ticker].to_numpy()
        e_low = sig.e_low[ticker].to_numpy()
        o_high = sig.o_high[ticker].to_numpy()
        o_low = sig.o_low[ticker].to_numpy()
        th_high = theta_high.get(ticker, np.nan)
        th_low = theta_low.get(ticker, np.nan)
        adj_open = panel.adj_open[ticker].to_numpy()
        adv = panel.adv[ticker].to_numpy()

        # NaN means "no threshold available, skip this side"; +/-inf is a
        # deliberate always-true gate (the Donchian twin) and must NOT be
        # treated as unavailable -- np.isfinite(-inf) is False, so the guard
        # here checks isnan specifically, not isfinite.
        if np.isnan(th_high) and np.isnan(th_low):
            continue

        long_trigger = (e_high == 1) & cmp(o_high, th_high) if not np.isnan(th_high) else np.zeros(T, dtype=bool)
        short_trigger = (e_low == 1) & cmp(o_low, th_low) if not np.isnan(th_low) else np.zeros(T, dtype=bool)

        for direction, trigger, opp_events, o_arr in (
            (+1, long_trigger, e_low, o_high),
            (-1, short_trigger, e_high, o_low),
        ):
            for p in np.flatnonzero(trigger):

```



```

        entry_pos = p + 1
        if entry_pos >= T:
            continue
        exit_pos, exit_reason = compute_exit(entry_pos, h, opp_events, T)
        entry_price, exit_price = adj_open[entry_pos], adj_open[exit_pos]
        if not (np.isfinite(entry_price) and np.isfinite(exit_price)) or exit_pos <= entry_pos:
            continue
        candidates.append(CandidateTrade(
            ticker=ticker, direction=direction,
            decision_date=calendar[p], entry_date=calendar[entry_pos], entry_pos=entry_pos,
            exit_date=calendar[exit_pos], exit_pos=exit_pos, exit_reason=exit_reason,
            entry_price=entry_price, exit_price=exit_price, o_value=float(o_arr[p]),
            entry_adv=adv[entry_pos - 1] if entry_pos > 0 else np.nan,
            exit_adv=adv[exit_pos - 1] if exit_pos > 0 else np.nan,
        ))
    return candidates

def admit_candidates(candidates, max_concurrent=config.MAX_CONCURRENT):
    """FIFO/one-per-asset/max-concurrent admission ledger. Deterministic
    tie-break among same-day competitors: higher occupation (stronger
    conviction) admitted first, then ticker alphabetically."""
    ordered = sorted(candidates, key=lambda c: (c.entry_pos, -c.o_value, c.ticker))
    heap = [] # (exit_pos, ticker)
    open_tickers = set()
    admitted = []
    for c in ordered:
        while heap and heap[0][0] <= c.entry_pos:
            _, ticker = heapq.heappop(heap)
            open_tickers.discard(ticker)
        if c.ticker in open_tickers:
            continue
        if len(heap) >= max_concurrent:
            continue
        heapq.heappush(heap, (c.exit_pos, c.ticker))
        open_tickers.add(c.ticker)
        admitted.append(c)
    return admitted

def assign_weights(panel, admitted, k, gross_cap=config.GROSS_CAP,
                   vol_lookback=config.SIZING_VOL_LOOKBACK):
    """Event-driven inverse-vol sizing:  $w_i = \text{direction} * k / \text{sigma\_hat}_i$ 
    ( $\text{sigma\_hat}_i$  annualized, measured through the day BEFORE entry). Gross
    exposure is only ever checked/enforced at entry events (no daily
    re-sizing); a new entry that would breach the gross cap is haircut down
    to whatever budget remains, existing open positions are never resized."""
    ordered = sorted(admitted, key=lambda c: c.entry_pos)
    heap = [] # (exit_pos, abs_weight)
    running_gross = 0.0
    trades = []
    ann = np.sqrt(config.TRADING_DAYS_YEAR)

    for c in ordered:
        while heap and heap[0][0] <= c.entry_pos:
            _, w_abs = heapq.heappop(heap)
            running_gross -= w_abs
        sv = panel.sizing_vol[c.ticker]
        sigma_daily = sv.iloc[c.entry_pos - 1] if c.entry_pos > 0 else np.nan
        sigma_ann = sigma_daily * ann if np.isfinite(sigma_daily) else np.nan
        w_abs_raw = inverse_vol_weight(sigma_ann, k)
        headroom = max(0.0, gross_cap - running_gross)
        w_abs = min(w_abs_raw, headroom)
        w = c.direction * w_abs
        running_gross += w_abs
        heapq.heappush(heap, (c.exit_pos, w_abs))

        gross_return = c.direction * (c.exit_price / c.entry_price - 1.0)
        cost_frac = trade_cost_fraction(c.entry_adv, c.exit_adv)
        net_pnl = w * gross_return - abs(w) * cost_frac
        trades.append(Trade(
            ticker=c.ticker, direction=c.direction, decision_date=c.decision_date,
            entry_date=c.entry_date, entry_pos=c.entry_pos, exit_date=c.exit_date,
            exit_pos=c.exit_pos, exit_reason=c.exit_reason, entry_price=c.entry_price,
            exit_price=c.exit_price, o_value=c.o_value, entry_adv=c.entry_adv, exit_adv=c.exit_adv,
            weight=w, sigma_hat=sigma_ann, gross_return=gross_return, cost_frac=cost_frac,
            net_pnl_contribution=net_pnl,
        ))
    return trades

def daily_returns(panel, trades) -> pd.Series:

```

```

"""Exact per-leg accounting: entry day is open->close, held days are
close->close, exit day is close[t-1]->open[t]; round-trip cost charged
on the entry day."""
calendar = panel.dates
T = len(calendar)
port_ret = np.zeros(T)
close_cache, open_cache = {}, {}

for tr in trades:
    if tr.ticker not in close_cache:
        close_cache[tr.ticker] = panel.adj_close[tr.ticker].to_numpy()
        open_cache[tr.ticker] = panel.adj_open[tr.ticker].to_numpy()
    aclose, aopen = close_cache[tr.ticker], open_cache[tr.ticker]
    p0, p1, w = tr.entry_pos, tr.exit_pos, tr.weight
    if p1 <= p0 or w == 0.0:
        continue
    if np.isfinite(aopen[p0]) and np.isfinite(aclose[p0]) and aopen[p0] != 0:
        port_ret[p0] += w * (aclose[p0] / aopen[p0] - 1.0)
    for t in range(p0 + 1, p1):
        if np.isfinite(aclose[t]) and np.isfinite(aclose[t - 1]) and aclose[t - 1] != 0:
            port_ret[t] += w * (aclose[t] / aclose[t - 1] - 1.0)
    if np.isfinite(aopen[p1]) and np.isfinite(aclose[p1 - 1]) and aclose[p1 - 1] != 0:
        port_ret[p1] += w * (aopen[p1] / aclose[p1 - 1] - 1.0)
    port_ret[p0] -= abs(w) * tr.cost_frac

return pd.Series(port_ret, index=calendar)

def run_backtest(panel, sig, theta_high, theta_low, h, k, gross_cap=config.GROSS_CAP,
                 max_concurrent=config.MAX_CONCURRENT, comparator="ge"):
    candidates = generate_candidates(panel, sig, theta_high, theta_low, h, comparator=comparator)
    admitted = admit_candidates(candidates, max_concurrent=max_concurrent)
    trades = assign_weights(panel, admitted, k, gross_cap=gross_cap)
    rets = daily_returns(panel, trades)
    return {"candidates": candidates, "admitted": admitted, "trades": trades, "returns": rets}

def annualized_vol(returns: pd.Series) -> float:
    return metrics.ann_vol(returns)

def _solve_k_for_target_vol(panel, admitted, is_start, is_end, target_vol, gross_cap,
                             k0=None, max_iter=25, tol=1e-4):
    """
    Iterative rescale to the target IS vol. A single-shot linear rescale
    ( $k_{final} = k0 * target\_vol(k0)$ ) is only exact when the gross cap never
    binds; when it does (e.g. the Donchian twin, which runs far more
    concurrent positions and saturates the 200% cap far more often), the
     $k \rightarrow vol$  relationship is no longer linear and a single shot overshoots.
    Iterating the same ratio-rescale to a fixed point corrects for this
    without assuming linearity -- it only needs the map to be locally
    well-behaved near the fixed point, not globally linear.
    """
    k = k0 if k0 is not None else target_vol
    vol = np.nan
    for _ in range(max_iter):
        trades = assign_weights(panel, admitted, k, gross_cap=gross_cap)
        rets = daily_returns(panel, trades)
        vol = annualized_vol(rets.loc[is_start:is_end])
        if not (np.isfinite(vol) and vol > 0):
            break
        rel_err = abs(vol - target_vol) / target_vol
        if rel_err < tol:
            break
        k = k * (target_vol / vol)
    return k, vol

def run_is_oos(panel, sig, theta_high, theta_low, h, is_start=config.IS_START, is_end=config.IS_END,
               target_vol=config.PORTFOLIO_VOL_TARGET, gross_cap=config.GROSS_CAP,
               max_concurrent=config.MAX_CONCURRENT, comparator="ge"):
    """
    Single continuous full-sample backtest (admission/FIFO never resets at
    the IS/OOS-1 boundary -- it's one book; data before is_start is a
    warm-up buffer only, present so the largest grid window (n=160) has a
    full rolling lookback by the first IS date), with the vol-target scalar
    k calibrated using ONLY the [is_start, is_end] slice of realized returns
    (iteratively, to converge despite the 200% gross cap's nonlinearity) and
    then frozen for the whole sample (so OOS-1 sizing never peeks at OOS-1
    vol). Returns full/IS/OOS-1 daily-return series plus the blotter.
    """
    candidates = generate_candidates(panel, sig, theta_high, theta_low, h, comparator=comparator)

```

```

admitted = admit_candidates(candidates, max_concurrent=max_concurrent)

k_final, vol_is_at_k0 = _solve_k_for_target_vol(panel, admitted, is_start, is_end, target_vol, gross_cap)

trades = assign_weights(panel, admitted, k_final, gross_cap=gross_cap)
rets = daily_returns(panel, trades)

return {
    "candidates": candidates, "admitted": admitted, "trades": trades,
    "returns_full": rets, "returns_is": rets.loc[is_start:is_end],
    "returns_oos1": rets.loc[is_end:].iloc[1:],
    "k": k_final, "vol_is_at_k0": vol_is_at_k0,
    "achieved_vol_is": annualized_vol(rets.loc[is_start:is_end]),
}

```

```

"""
Dammluckan -- Donchian twin and anti-twin comparison strategies.

Donchian twin: identical record events (E+/E-), same exits/sizing/portfolio
machinery, but WITHOUT the occupation gate (theta = -inf, i.e. every record
event is traded). This is the naive Donchian/52-week-breakout projection of
the hypothesis -- already-known ground CTA capital harvests on a monthly
horizon -- and Dammluckan must beat it net, not just beat a flat benchmark
("nettoöverskott mot Donchian-tvillingen <= 0" is a pre-registered kill
criterion).

Anti-twin: same record events, gated the OPPOSITE way -- entries require LOW
occupation (0 <= the mirrored low percentile of the same null, e.g. the 20th
when the primary cell uses the 80th). The hypothesis requires this twin to
carry ~zero excess return: unconditional/low-occupation breaks bear nothing,
high-occupation wall-breaks bear all of it.
"""
import numpy as np
import pandas as pd

from . import backtest

def donchian_twin_thetas(tickers):
    """theta = -inf on both sides: every record event is admitted, i.e. the
    occupation gate is switched off entirely."""
    neg_inf = pd.Series(-np.inf, index=list(tickers))
    return neg_inf, neg_inf

def run_donchian_twin(panel, sig, h):
    th_high, th_low = donchian_twin_thetas(panel.tickers)
    return backtest.run_is_oos(panel, sig, th_high, th_low, h, comparator="ge")

def run_anti_twin(panel, sig, anti_theta_high, anti_theta_low, h):
    return backtest.run_is_oos(panel, sig, anti_theta_high, anti_theta_low, h, comparator="le")

```

```

"""
Dammluckan -- TSMOM (time-series momentum) proxy.

Adapted from research/formdriften/tsmom.py: this is "the TSMOM-proxyn
(dokumenterad i repot)" the brief refers to -- a mechanical, dependency-free
Moskowitz-Ooi-Pedersen-style construction with no fitted parameters, used
both as (a) a redundancy-screen regressor and (b) the  $p(\text{TSMOM}) > 0.4$ 
diversification kill criterion, computed on the same panel/universe the
candidate strategy trades.
"""
import numpy as np
import pandas as pd

from . import config

LOOKBACK_DAYS = 252
VOL_WINDOW = 63
MAX_WEIGHT = 0.15

def month_end_dates(index: pd.DatetimeIndex) -> pd.DatetimeIndex:
    s = pd.Series(index, index=index)
    return pd.DatetimeIndex(s.groupby([index.year, index.month]).last().values)

def tsmom_returns(panel, lookback=LOOKBACK_DAYS, vol_window=VOL_WINDOW, max_weight=MAX_WEIGHT,
                  rebalance_dates=None) -> pd.Series:
    """
    Monthly-rebalanced TSMOM proxy: sign(trailing 12m adjusted-close return),
    inverse-63d-vol sized, 15% per-name cap, gross renormalized to 1.0 after
    clipping. Daily weights held flat between rebalance dates (signal decided
    strictly through the rebalance date's close, applied from the next day).
    """
    returns = panel.log_returns
    idx = returns.index
    if rebalance_dates is None:
        rebalance_dates = month_end_dates(idx)

    vol = returns.rolling(vol_window, min_periods=vol_window // 2).std(ddof=1)
    daily_w = pd.DataFrame(0.0, index=idx, columns=returns.columns)

    px = panel.adj_close
    for k, dt in enumerate(rebalance_dates):
        pos = idx.searchsorted(dt)
        if pos < lookback:
            continue
        trail_start = idx[pos - lookback]
        p0 = px.loc[trail_start]
        p1 = px.loc[dt]
        trail_ret = p1 / p0 - 1.0
        sig = np.sign(trail_ret).reindex(returns.columns)

        v = vol.loc[dt].reindex(returns.columns)
        raw = (sig / v).replace([np.inf, -np.inf], np.nan).fillna(0.0)
        gross = raw.abs().sum()
        w = raw / gross if gross > 0 else raw * 0.0
        w = w.clip(lower=-max_weight, upper=max_weight)
        gross2 = w.abs().sum()
        if gross2 > 0:
            w = w / gross2

        end_dt = rebalance_dates[k + 1] if k + 1 < len(rebalance_dates) else idx[-1]
        mask = (idx > dt) & (idx <= end_dt)
        daily_w.loc[mask, :] = w.values

    # daily_w[d] already holds the weight decided at the most recent
    # rebalance date strictly before d (mask excludes the rebalance day
    # itself), so no additional lag is applied here -- matching Formdriften's
    # tsmom.py execution-timing convention exactly.
    port_ret = (daily_w.fillna(0.0) * returns.fillna(0.0)).sum(axis=1)
    return port_ret

```

```

"""
Damluckan -- portfolio-level null batteries #3 and #4 from the
pre-registered "Nollhypotes-baslinje":

(3) Block-permuterade avkastningar per tillgang -> the FULL rule set
(frozen n/c/theta_i, same h, same k) re-run on synthetic per-asset
block-bootstrapped price paths.
(4) Slumpade entrytidpunkter, matched in count and asset -- same exit
rule, same sizing/portfolio machinery, only WHEN each asset enters is
randomized.

Both produce a null distribution of portfolio Sharpe (per-period) used both
as a standalone significance test and as an additional DSR trial-pool input.
"""
import numpy as np
import pandas as pd

from . import config
from . import data as data_mod
from . import nulls
from . import signal as signal_mod
from . import backtest
from . import metrics

def _synthetic_full_panel(panel, rng, block=config.BLOCK_LENGTH):
    """Independently block-bootstrap every ticker's own return history,
    preserving each ticker's own real NaN-prefix (inception date) so a
    late-listed name (HYG, GLD, UUP, ...) doesn't get more synthetic
    tradeable history than it really had. adj_open is set equal to
    adj_close (declared simplification: open/close micro-gaps are not part
    of what's being tested here) and ADV/volume are carried over
    POSITIONALLY from the real panel (the null targets return structure,
    not the liquidity/cost environment)."""
    T = len(panel.dates)
    cols = {}
    for ticker in panel.tickers:
        real_col = panel.raw_close[ticker]
        n_valid = int(real_col.notna().sum())
        if n_valid < 30:
            continue
        r = nulls._asset_log_returns(panel, ticker)
        path = nulls.synthetic_price_path(r, n_valid - 1, rng, block=block) # length n_valid
        cols[ticker] = np.concatenate([np.full(T - n_valid, np.nan), path])
    if not cols:
        return None
    raw_close = pd.DataFrame(cols, index=panel.dates)
    synth = data_mod.Panel(
        tickers=panel.tickers, raw_close=raw_close, raw_open=raw_close,
        high=raw_close, low=raw_close, adj_close=raw_close, adj_open=raw_close,
        volume=panel.volume.reindex(columns=raw_close.columns),
    )
    # Reuse real dollar_volume/adv positionally (liquidity/cost environment
    # held fixed; only return structure is randomized).
    synth.dollar_volume = panel.dollar_volume.reindex(columns=raw_close.columns)
    synth.adv = panel.adv.reindex(columns=raw_close.columns)
    return synth

def block_permuted_returns_null(panel, n, c, theta_high, theta_low, h, k, n_draws=config.N_BLOCK_DRAWS,
                                seed=0, comparator="ge"):
    """Null twin #3: re-run the FULL frozen rule set on synthetic,
    independently-block-bootstrapped per-asset price paths."""
    rng = np.random.default_rng(seed)
    sharpes, ann_returns = [], []
    for draw in range(n_draws):
        synth = _synthetic_full_panel(panel, rng)
        if synth is None:
            continue
        sig = signal_mod.build_signal(synth, n=n, c=c)
        res = backtest.run_backtest(synth, sig, theta_high, theta_low, h, k=k, comparator=comparator)
        r = res["returns"]
        sharpes.append(metrics.sharpe(r) / np.sqrt(config.TRADING_DAYS_YEAR)) # per-period Sharpe
        ann_returns.append(metrics.ann_return(r))
    return {"sharpes_per_period": np.array(sharpes), "ann_returns": np.array(ann_returns), "n_draws": len(sharpes)}

def randomized_entry_null(panel, sig, admitted_real, h, k, gross_cap=config.GROSS_CAP,

```

```

max_concurrent=config.MAX_CONCURRENT, n_draws=config.N_BLOCK_DRAWS, seed=0):
    """Null twin #4: same per-(ticker,direction) trade COUNT as the real
    admitted set, but uniformly random entry decision dates; same exit rule
    (time-stop / first opposite record on the REAL asset's own signal),
    same sizing/portfolio machinery."""
    from collections import Counter
    from .portfolio import CandidateTrade

    calendar = panel.dates
    T = len(calendar)
    counts = Counter((c.ticker, c.direction) for c in admitted_real)

    eligible_pos = {}
    for ticker in panel.tickers:
        valid = np.flatnonzero(panel.raw_close[ticker].notna().to_numpy())
        eligible_pos[ticker] = valid[(valid >= config.VOL_LOOKBACK) & (valid < T - 1)]

    rng = np.random.default_rng(seed)
    sharpes, ann_returns = [], []
    for draw in range(n_draws):
        candidates = []
        for (ticker, direction), m in counts.items():
            pool = eligible_pos[ticker]
            if len(pool) == 0 or m == 0:
                continue
            decision_pos = rng.choice(pool, size=min(m, len(pool)), replace=False)
            opp_events = (panel_e_low(sig, ticker) if direction == 1 else panel_e_high(sig, ticker))
            adj_open = panel.adj_open[ticker].to_numpy()
            adv = panel.adv[ticker].to_numpy()
            for p in decision_pos:
                entry_pos = p + 1
                if entry_pos >= T:
                    continue
                exit_pos, exit_reason = backtest.compute_exit(entry_pos, h, opp_events, T)
                entry_price, exit_price = adj_open[entry_pos], adj_open[exit_pos]
                if not (np.isfinite(entry_price) and np.isfinite(exit_price)) or exit_pos <= entry_pos:
                    continue
                candidates.append(CandidateTrade(
                    ticker=ticker, direction=direction, decision_date=calendar[p],
                    entry_date=calendar[entry_pos], entry_pos=entry_pos, exit_date=calendar[exit_pos],
                    exit_pos=exit_pos, exit_reason=exit_reason, entry_price=entry_price,
                    exit_price=exit_price, o_value=0.0,
                    entry_adv=adv[entry_pos - 1] if entry_pos > 0 else np.nan,
                    exit_adv=adv[exit_pos - 1] if exit_pos > 0 else np.nan,
                ))
        admitted = backtest.admit_candidates(candidates, max_concurrent=max_concurrent)
        trades = backtest.assign_weights(panel, admitted, k, gross_cap=gross_cap)
        r = backtest.daily_returns(panel, trades)
        sharpes.append(metrics.sharpe(r) / np.sqrt(config.TRADING_DAYS_YEAR))
        ann_returns.append(metrics.ann_return(r))
    return {"sharpes_per_period": np.array(sharpes), "ann_returns": np.array(ann_returns), "n_draws": len(sharpes)}

def panel_e_low(sig, ticker):
    return sig.e_low[ticker].to_numpy()

def panel_e_high(sig, ticker):
    return sig.e_high[ticker].to_numpy()

def null_pvalue(real_stat, null_stats) -> float:
    null_stats = np.asarray(null_stats, dtype=float)
    null_stats = null_stats[np.isfinite(null_stats)]
    if not np.isfinite(real_stat) or len(null_stats) == 0:
        return np.nan
    return (np.sum(null_stats >= real_stat) + 1) / (len(null_stats) + 1)

```

```

"""
Dammluckan -- Steg 1 estimator-level checks: event-level IC test (vs. block
null) and the mandatory redundancy screen (0+ / 0- vs. vol, skew, |r|-
autocorrelation, n-day momentum, distance-to-max, and the TSMOM proxy).

This operates one level below the portfolio backtest: it asks whether
occupation, AT THE MOMENT OF AN UNCONDITIONAL RECORD BREAK, predicts what
happens over the next h trading days -- decoupled from FIFO/gross-cap
portfolio mechanics, which is exactly what "estimatornull före
portföljbygge" (estimator null before portfolio construction) means.
"""

import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy import stats as scipy_stats

from . import config
from . import metrics
from . import nulls

def _control_frame(panel, n):
    """Side-agnostic market-state controls, evaluated through close t."""
    vol = panel.band_vol
    ret = panel.log_returns
    skew = ret.rolling(config.VOL_LOOKBACK, min_periods=config.VOL_MIN_PERIODS).skew()
    abs_r = ret.abs()
    autocorr = abs_r.rolling(config.VOL_LOOKBACK, min_periods=config.VOL_MIN_PERIODS).apply(
        lambda w: pd.Series(w).autocorr(lag=1), raw=False
    )
    momentum_n = panel.adj_close / panel.adj_close.shift(n) - 1.0
    tsmom_proxy = panel.adj_close / panel.adj_close.shift(252) - 1.0
    return {"vol": vol, "skew": skew, "autocorr_abs_r": autocorr,
            "momentum_n": momentum_n, "tsmom_proxy": tsmom_proxy}

def build_event_table(panel, sig, h, side, controls=None) -> pd.DataFrame:
    """One row per (ticker, event date) where E+ (side='high') / E- (side='low')
    fires -- unconditional on occupation. Columns: o, fwd_return (signed by
    side), dist_to_max, plus the redundancy-screen controls IF a
    precomputed 'controls' dict (from _control_frame) is passed -- kept
    optional because _control_frame's autocorrelation term is expensive and
    must not be recomputed inside the (hundreds-of-draws) null-test loop,
    which only needs o/fwd_return."""
    e = sig.e_high if side == "high" else sig.e_low
    o = sig.o_high if side == "high" else sig.o_low
    m = sig.m_high if side == "high" else sig.m_low
    direction = 1.0 if side == "high" else -1.0
    px = panel.adj_close
    raw_px = panel.raw_close

    rows = []
    T = len(panel.dates)
    for ticker in panel.tickers:
        e_arr = e[ticker].to_numpy()
        hit_pos = np.flatnonzero(e_arr == 1)
        if not len(hit_pos):
            continue
        o_arr = o[ticker].to_numpy()
        px_arr = px[ticker].to_numpy()
        raw_arr = raw_px[ticker].to_numpy()
        m_arr = m[ticker].to_numpy()
        ctrl_arrs = {k: v[ticker].to_numpy() for k, v in controls.items()} if controls else {}
        for p in hit_pos:
            if p + h >= T or not np.isfinite(o_arr[p]):
                continue
            fwd = direction * (px_arr[p + h] / px_arr[p] - 1.0) if np.isfinite(px_arr[p]) and px_arr[p] != 0 else
np.nan
            dist_to_max = (raw_arr[p] - m_arr[p]) / m_arr[p] if np.isfinite(m_arr[p]) and m_arr[p] != 0 else np.nan
            row = {"ticker": ticker, "date": panel.dates[p], "o": o_arr[p], "fwd_return": fwd,
                  "dist_to_max": dist_to_max}
            for k, arr in ctrl_arrs.items():
                row[k] = arr[p]
            rows.append(row)
    return pd.DataFrame(rows)

def event_level_ic(panel, sig, h, side="both") -> dict:

```



```

"""Spearman IC of occupation vs. signed h-day forward return, pooled
across all (ticker, event-date) rows, unconditional on occupation."""
tables = []
if side in ("high", "both"):
    tables.append(build_event_table(panel, sig, h, "high"))
if side in ("low", "both"):
    tables.append(build_event_table(panel, sig, h, "low"))
df = pd.concat(tables, ignore_index=True) if tables else pd.DataFrame()
df = df.dropna(subset=["o", "fwd_return"])
ic = metrics.information_coefficient(df["o"], df["fwd_return"]) if len(df) >= 5 else np.nan
return {"ic": ic, "n_events": len(df), "table": df}

def ic_null_test(panel, sig, h, n_draws=config.N_BLOCK_DRAWS, seed=0) -> dict:
    """Block-bootstrap null for the pooled event-level IC: for each draw,
resample every asset's own IS return history (circular block bootstrap),
rebuild E/O/forward-return on the synthetic path with the SAME (n, c),
recompute the pooled IC, and derive a two-sided empirical p-value."""
    from . import signal as signal_mod

    real = event_level_ic(panel, sig, h, side="both")
    real_ic = real["ic"]

    rng = np.random.default_rng(seed)
    null_ics = []
    for draw in range(n_draws):
        synth_cols = {}
        for ticker in panel.tickers:
            r = nulls._asset_log_returns(panel, ticker)
            if len(r) <= sig.n + config.VOL_LOOKBACK + h:
                continue
            path = nulls.synthetic_price_path(r, len(r), rng, block=config.BLOCK_LENGTH)
            synth_cols[ticker] = pd.Series(path[: len(panel.dates)], index=panel.dates[: len(path)])
        if not synth_cols:
            continue
        synth_df = pd.DataFrame(synth_cols).reindex(panel.dates)
        synth_panel = _wrap_synthetic_panel(panel, synth_df)
        synth_sig = signal_mod.build_signal(synth_panel, n=sig.n, c=sig.c)
        res = event_level_ic(synth_panel, synth_sig, h, side="both")
        if np.isfinite(res["ic"]):
            null_ics.append(res["ic"])

    null_ics = np.array(null_ics)
    if np.isfinite(real_ic) and len(null_ics):
        pval = (np.sum(np.abs(null_ics) >= abs(real_ic)) + 1) / (len(null_ics) + 1)
    else:
        pval = np.nan
    return {"real_ic": real_ic, "null_ics": null_ics, "p_value": pval, "n_draws": len(null_ics)}

def _wrap_synthetic_panel(panel, synth_raw_close):
    """Build a lightweight Panel-like object around a synthetic raw-close
path, reusing the real panel's adjustment-factor structure isn't
meaningful for synthetic data, so adj_close == raw_close and
adj_open == raw_close (no separate open series needed for the IC test,
which only touches raw_close/adj_close/dates)."""
    from . import data as data_mod

    return data_mod.Panel(
        tickers=panel.tickers,
        raw_close=synth_raw_close, raw_open=synth_raw_close,
        high=synth_raw_close, low=synth_raw_close,
        adj_close=synth_raw_close, adj_open=synth_raw_close,
        volume=pd.DataFrame(np.nan, index=synth_raw_close.index, columns=synth_raw_close.columns),
    )

def redundancy_regression(panel, sig, h) -> dict:
    """Pooled OLS: 0 ~ vol + skew + |r|-autocorr + n-day-momentum +
distance-to-max + TSMOM-proxy. Kill if R^2 > 0.5 (0 is redundant with
already-known state variables)."""
    controls_frame = _control_frame(panel, sig.n)
    tables = [build_event_table(panel, sig, h, "high", controls=controls_frame),
              build_event_table(panel, sig, h, "low", controls=controls_frame)]
    df = pd.concat(tables, ignore_index=True)
    controls = ["vol", "skew", "autocorr_abs_r", "momentum_n", "dist_to_max", "tsmom_proxy"]
    df = df.dropna(subset=["o"] + controls)
    if len(df) < 30:
        return {"r2": np.nan, "n_obs": len(df), "spearman": {}}

    X = sm.add_constant(df[controls].to_numpy())
    y = df["o"].to_numpy()

```

```
model = sm.OLS(y, X).fit()
r2 = model.rsquared

spearman = {}
for c in controls:
    rho, _ = scipy_stats.spearmanr(df["o"], df[c])
    spearman[c] = rho

return {"r2": r2, "n_obs": len(df), "spearman": spearman, "params": dict(zip(["const"] + controls, model.params))}
```

```

"""
Daml luckan -- performance/risk statistics.

DSR/PSR follow the Bailey & Lopez de Prado (2014) formulation, which is
copy-structurally identical across every sibling research branch in this
repo (fasflocken/stats.py, oglegrinden/stats.py, irreversibility_lab/
validation.py, formdriften/metrics.py) -- reproduced here in the same form.
Newey-West t-stats use statsmodels HAC, matching oglegrinden/formdriften.
"""

import numpy as np
import pandas as pd
from scipy import stats as scipy_stats
import statsmodels.api as sm

from . import config

TRADING_DAYS = config.TRADING_DAYS_YEAR
EULER_MASCHERONI = 0.5772156649015329

def ann_return(returns: pd.Series) -> float:
    r = returns.dropna()
    return r.mean() * TRADING_DAYS if len(r) else np.nan

def ann_vol(returns: pd.Series) -> float:
    r = returns.dropna()
    return r.std(ddof=1) * np.sqrt(TRADING_DAYS) if len(r) > 1 else np.nan

def sharpe(returns: pd.Series) -> float:
    r = returns.dropna()
    if len(r) < 2 or r.std(ddof=1) == 0:
        return np.nan
    return r.mean() / r.std(ddof=1) * np.sqrt(TRADING_DAYS)

def max_drawdown(returns: pd.Series) -> float:
    r = returns.dropna()
    if not len(r):
        return np.nan
    curve = (1.0 + r).cumprod()
    dd = curve / curve.cummax() - 1.0
    return dd.min()

def newey_west_tstat(returns: pd.Series, lags=None) -> float:
    """HAC-robust t-stat that the mean (per-period) return is 0."""
    r = returns.dropna()
    if len(r) < 5:
        return np.nan
    if lags is None:
        lags = int(np.floor(4 * (len(r) / 100) ** (2 / 9)))
    ones = np.ones(len(r))
    model = sm.OLS(r.to_numpy(), ones).fit(cov_type="HAC", cov_kws={"maxlags": max(lags, 1)})
    return float(model.tvalues[0])

def probabilistic_sharpe_ratio(sr_hat: float, benchmark_sr: float, n_obs: int,
                               skew: float = 0.0, kurtosis: float = 3.0) -> float:
    """PSR (Bailey & Lopez de Prado 2012). sr_hat/benchmark_sr must be on the
    SAME (per-period, unannualized) scale."""
    if n_obs < 2:
        return np.nan
    excess_kurt = kurtosis - 3.0
    denom = np.sqrt(max(1e-12, 1 - skew * sr_hat + (excess_kurt / 4.0) * sr_hat ** 2))
    z = (sr_hat - benchmark_sr) * np.sqrt(n_obs - 1) / denom
    return float(scipy_stats.norm.cdf(z))

def expected_max_sharpe(sr_trials) -> float:
    """E[max Sharpe] under N independent noise trials (extreme-value
    approximation), on the per-period scale of sr_trials."""
    sr_trials = np.asarray(sr_trials, dtype=float)
    sr_trials = sr_trials[np.isfinite(sr_trials)]
    n = len(sr_trials)
    if n < 2:
        raise ValueError("expected_max_sharpe requires at least 2 trials")

```

```

sigma_sr = np.std(sr_trials, ddof=1)
return sigma_sr * ((1 - EULER_MASCHERONI) * scipy_stats.norm.ppf(1 - 1.0 / n)
                  + EULER_MASCHERONI * scipy_stats.norm.ppf(1 - 1.0 / (n * np.e)))

def deflated_sharpe_ratio(sr_hat: float, n_obs: int, sr_trials, skew: float = 0.0,
                          kurtosis: float = 3.0) -> dict:
    """Returns both the DSR probability (PSR against the expected-max-under-
    the-null benchmark) and the Sharpe-scaled excess gap (observed -
    E[max under null]), since a bare "DSR <= 0" rule needs the gap, not the
    probability. sr_hat and sr_trials must both be per-period Sharpes."""
    sr0 = expected_max_sharpe(sr_trials)
    psr = probabilistic_sharpe_ratio(sr_hat, sr0, n_obs, skew, kurtosis)
    return {"dsr_prob": psr, "dsr_excess": sr_hat - sr0, "expected_max_sr": sr0}

def information_coefficient(signal_vals, forward_returns) -> float:
    """Spearman rank correlation between a signal and forward returns."""
    df = pd.concat([pd.Series(signal_vals), pd.Series(forward_returns)], axis=1).dropna()
    if len(df) < 5:
        return np.nan
    rho, _ = scipy_stats.spearmanr(df.iloc[:, 0], df.iloc[:, 1])
    return rho

def pnl_quarter_concentration(returns: pd.Series) -> float:
    """Max fraction of total |PnL| (sum of returns) from a single calendar
    quarter -- ported from formdriften/metrics.py::pnl_quarter_concentration."""
    r = returns.dropna()
    if not len(r):
        return np.nan
    q = r.groupby([r.index.year, r.index.quarter]).sum()
    total = q.abs().sum()
    return float(q.abs().max() / total) if total != 0 else np.nan

def sub_period_boundaries(start, end, n_periods=config.N_SUBPERIODS):
    start, end = pd.Timestamp(start), pd.Timestamp(end)
    edges = pd.date_range(start, end, periods=n_periods + 1)
    return [(edges[i], edges[i + 1]) for i in range(n_periods)]

def sub_period_sign_consistency(returns: pd.Series, start, end, n_periods=config.N_SUBPERIODS) -> dict:
    """Sign of annualized return in each of n_periods equal calendar
    sub-periods; kill criterion is inconsistency in >= 2 of n_periods
    relative to the full-sample sign."""
    full_sign = np.sign(ann_return(returns))
    boundaries = sub_period_boundaries(start, end, n_periods)
    signs, ann_rets = [], []
    for lo, hi in boundaries:
        sub = returns.loc[lo:hi]
        ar = ann_return(sub)
        ann_rets.append(ar)
        signs.append(np.sign(ar) if np.isfinite(ar) else 0.0)
    n_inconsistent = sum(1 for s in signs if s != 0 and s != full_sign)
    return {"full_sign": full_sign, "period_signs": signs, "period_ann_returns": ann_rets,
            "n_inconsistent": n_inconsistent, "boundaries": boundaries}

def event_concentration(trades, tickers) -> dict:
    """Step 0: per-side event-count and max-single-asset-share check."""
    from collections import Counter
    counts = Counter(t.ticker for t in trades)
    total = sum(counts.values())
    max_share = (max(counts.values()) / total) if total else np.nan
    max_ticker = max(counts, key=counts.get) if counts else None
    full_counts = {t: counts.get(t, 0) for t in tickers}
    return {"n_events": total, "counts": full_counts, "max_share": max_share, "max_ticker": max_ticker}

```

```

"""
Dammluckan -- 27-variant pre-registered grid (n x theta_pctl x h), DSR
deflation, and the neighborhood sign-majority robustness check.
"""
import itertools
import numpy as np
import pandas as pd

from . import config
from . import signal as signal_mod
from . import backtest
from . import metrics

def run_grid(panel, calib, is_end=config.IS_END, target_vol=config.PORTFOLIO_VOL_TARGET):
    """Runs all 27 (n, theta_pctl, h) cells. Signal (which depends only on n)
    is built once per n and reused across the 9 (theta_pctl, h) sub-cells
    that share it."""
    rows = []
    cell_returns = {}
    for n in config.N_GRID:
        c = calib["c"][n]
        sig = signal_mod.build_signal(panel, n=n, c=c)
        for pctl in config.THETA_PCTL_GRID:
            th_high = calib["theta_high"][n][pctl]
            th_low = calib["theta_low"][n][pctl]
            for h in config.H_GRID:
                res = backtest.run_is_oos(panel, sig, th_high, th_low, h, is_end=is_end, target_vol=target_vol)
                cell_id = f"{n}_{pctl}_{h}"
                is_sharpe = metrics.sharpe(res["returns_is"])
                oos_sharpe = metrics.sharpe(res["returns_oos1"])
                rows.append({
                    "cell_id": cell_id, "n": n, "theta_pctl": pctl, "h": h,
                    "is_sharpe": is_sharpe, "oos1_sharpe": oos_sharpe,
                    "is_sharpe_pp": is_sharpe / np.sqrt(config.TRADING_DAYS_YEAR) if np.isfinite(is_sharpe) else
np.nan,
                    "oos1_sharpe_pp": oos_sharpe / np.sqrt(config.TRADING_DAYS_YEAR) if np.isfinite(oos_sharpe) else
np.nan,
                    "n_trades": len(res["trades"]),
                    "n_trades_is": sum(1 for t in res["trades"]
                                     if pd.Timestamp(config.IS_START) <= t.entry_date <= pd.Timestamp(is_end)),
                    "is_primary": (n == config.N_DEFAULT and pctl == config.THETA_PCTL_DEFAULT and h ==
config.H_DEFAULT),
                })
                cell_returns[cell_id] = res["returns_full"]
    table = pd.DataFrame(rows)
    return {"table": table, "returns_by_cell": cell_returns}

def neighborhood_isolation_check(table: pd.DataFrame, target_cell_id: str, sharpe_col="is_sharpe",
                                neighbor_frac_threshold=0.5) -> dict:
    """Flags a lone-winner fluke: among cells that differ from the target in
    exactly ONE grid axis (n, theta_pctl, or h), what fraction share the
    target's Sharpe SIGN? A real effect should show a sign majority in its
    immediate neighborhood, not an isolated spike."""
    row = table.loc[table.cell_id == target_cell_id].iloc[0]
    target_sign = np.sign(row[sharpe_col])
    neighbors = table[
        ((table.n == row.n) & (table.theta_pctl == row.theta_pctl) & (table.h != row.h)) |
        ((table.n == row.n) & (table.h == row.h) & (table.theta_pctl != row.theta_pctl)) |
        ((table.theta_pctl == row.theta_pctl) & (table.h == row.h) & (table.n != row.n))
    ]
    same_sign = (np.sign(neighbors[sharpe_col]) == target_sign).sum()
    frac = same_sign / len(neighbors) if len(neighbors) else np.nan
    return {"target_sign": target_sign, "n_neighbors": len(neighbors), "n_same_sign": int(same_sign),
            "frac_same_sign": frac, "passes": (frac >= neighbor_frac_threshold) if np.isfinite(frac) else False}

def dsr_from_grid(table: pd.DataFrame, returns_by_cell: dict, primary_cell_id: str,
                  period="is") -> dict:
    """DSR of the primary cell's Sharpe against the 27-cell grid as the
    trial pool (per-period Sharpes throughout, matching every sibling
    branch's own explicit convention)."""
    sharpe_col = f"{period}_sharpe_pp"
    trials = table[sharpe_col].dropna().to_numpy()
    primary_row = table.loc[table.cell_id == primary_cell_id].iloc[0]
    sr_hat = primary_row[sharpe_col]
    key = "returns_is" if period == "is" else "returns_oos1"

```

```

r = returns_by_cell[primary_cell_id]
if period == "is":
    r = r.loc[:config.IS_END]
else:
    r = r.loc[config.OOS1_START:]
r = r.dropna()
n_obs = len(r)
skew = r.skew() if n_obs > 2 else 0.0
kurt = r.kurtosis() + 3.0 if n_obs > 2 else 3.0 # pandas kurtosis is excess; DSR wants Pearson convention
return metrics.deflated_sharpe_ratio(sr_hat, n_obs, trials, skew=skew, kurtosis=kurt)

```

```

"""
Damm Luckan -- Step 0 of the pipeline: calibrate c (band constant, per grid n)
and theta_i (per-asset occupation threshold, per grid n / side / percentile),
IS-only (2004-2017), and freeze the result to output/calibration.pkl for
every other stage (backtest, grid search, OOS-1, OOS-2) to load and reuse
without ever re-touching OOS data during calibration.
"""
import os
import pickle
import time

from . import config
from . import data
from . import signal

HERE = os.path.dirname(os.path.abspath(__file__))
OUT_PATH = os.path.join(HERE, "output", "calibration.pkl")

def run(n_draws_c=200, n_draws_theta=config.N_THETA_CALIB_DRAWS, force=False):
    if os.path.exists(OUT_PATH) and not force:
        with open(OUT_PATH, "rb") as f:
            return pickle.load(f)

    t0 = time.time()
    panel = data.load_primary_panel()
    result = {"c": {}, "theta_high": {}, "theta_low": {}, "anti_theta_high": {}, "anti_theta_low": {}}

    # Anti-twin uses the mirrored LOW percentile (100-p) of the exact same
    # null distribution -- computed in the same pass as the primary
    # percentiles below, no extra bootstrap draws needed.
    anti_pctls = [100 - p for p in config.THETA_PCTL_GRID]

    for n in config.N_GRID:
        c = signal.calibrate_band_constant(panel, n=n, n_draws=n_draws_c, seed=42)
        result["c"][n] = c
        print(f"[{time.time()-t0:7.1f}s] n={n}: calibrated c={c:.4f}")

        all_high = signal.calibrate_theta_multi(
            panel, n=n, c=c, pctls=list(config.THETA_PCTL_GRID) + anti_pctls, side="high",
            n_draws=n_draws_theta, seed=100,
        )
        result["theta_high"][n] = {p: all_high[p] for p in config.THETA_PCTL_GRID}
        result["anti_theta_high"][n] = {p: all_high[100 - p] for p in config.THETA_PCTL_GRID}
        print(f"[{time.time()-t0:7.1f}s] n={n}: theta_high calibrated "
              f"(median theta80={all_high[80].median():.4f}, anti theta20={all_high[20].median():.4f})")

        all_low = signal.calibrate_theta_multi(
            panel, n=n, c=c, pctls=list(config.THETA_PCTL_GRID) + anti_pctls, side="low",
            n_draws=n_draws_theta, seed=200,
        )
        result["theta_low"][n] = {p: all_low[p] for p in config.THETA_PCTL_GRID}
        result["anti_theta_low"][n] = {p: all_low[100 - p] for p in config.THETA_PCTL_GRID}
        print(f"[{time.time()-t0:7.1f}s] n={n}: theta_low calibrated "
              f"(median theta80={all_low[80].median():.4f}, anti theta20={all_low[20].median():.4f})")

    os.makedirs(os.path.dirname(OUT_PATH), exist_ok=True)
    with open(OUT_PATH, "wb") as f:
        pickle.dump(result, f)
    print(f"[{time.time()-t0:7.1f}s] Saved calibration to {OUT_PATH}")
    return result

if __name__ == "__main__":
    run(force=True)

```

```

"""
Dammluckan -- full research pipeline orchestration.

Stages (run in order, each cached to output/<stage>.pkl so re-runs are
resumable): calibration -> gate0 (event count + concentration) -> gate1
(estimator-level IC null test + redundancy screen) -> primary (IS/00S-1
backtest of the primary cell, Donchian twin, anti-twin, null batteries #3/#4,
DSR, sub-period sign consistency, PnL concentration, TSMOM correlation) ->
grid (27-variant DSR deflation + neighborhood check) -> oos2 (secondary,
16-country-ETF confirmation surface, touched exactly once, using every
parameter frozen from the primary-universe IS calibration) -> verdict.
"""

import os
import pickle
import time
import sys

import numpy as np
import pandas as pd

from . import config
from . import data
from . import signal as signal_mod
from . import backtest
from . import twins
from . import battery
from . import robustness
from . import metrics
from . import grid as grid_mod
from . import tsmom
from . import run_calibration

HERE = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(HERE, "output")

def _cache_path(stage):
    return os.path.join(OUT_DIR, f"stage_{stage}.pkl")

def _save(stage, obj):
    os.makedirs(OUT_DIR, exist_ok=True)
    with open(_cache_path(stage), "wb") as f:
        pickle.dump(obj, f)

def _load(stage):
    path = _cache_path(stage)
    if os.path.exists(path):
        with open(path, "rb") as f:
            return pickle.load(f)
    return None

def _log(t0, msg):
    print(f"[{time.time()-t0:7.1f}s] {msg}")
    sys.stdout.flush()

PRIMARY_CELL = dict(n=config.N_DEFAULT, pctl=config.THETA_PCTL_DEFAULT, h=config.H_DEFAULT)

def stage_gate0(panel, calib, t0, force=False):
    if not force and (cached := _load("gate0")) is not None:
        return cached
    n, pctl, h = PRIMARY_CELL["n"], PRIMARY_CELL["pctl"], PRIMARY_CELL["h"]
    c = calib["c"][n]
    sig = signal_mod.build_signal(panel, n=n, c=c)
    th_high, th_low = calib["theta_high"][n][pctl], calib["theta_low"][n][pctl]
    candidates = backtest.generate_candidates(panel, sig, th_high, th_low, h)
    long_candidates = [c_ for c_ in candidates if c_.direction == 1]
    short_candidates = [c_ for c_ in candidates if c_.direction == -1]
    conc_long = metrics.event_concentration(long_candidates, panel.tickers)
    conc_short = metrics.event_concentration(short_candidates, panel.tickers)
    result = {
        "n_long": len(long_candidates), "n_short": len(short_candidates),
        "conc_long": conc_long, "conc_short": conc_short,
        "pass_count": len(long_candidates) >= config.MIN_EVENTS_PER_SIDE
    }

```



```

        and len(short_candidates) >= config.MIN_EVENTS_PER_SIDE,
        "pass_concentration": conc_long["max_share"] <= config.MAX_SINGLE_ASSET_EVENT_SHARE
        and conc_short["max_share"] <= config.MAX_SINGLE_ASSET_EVENT_SHARE,
    }
    _log(t0, f"gate0: n_long={result['n_long']} n_short={result['n_short']} "
           f"max_share_long={conc_long['max_share']:.3f} ({conc_long['max_ticker']}) "
           f"max_share_short={conc_short['max_share']:.3f} ({conc_short['max_ticker']}) "
           f"pass_count={result['pass_count']} pass_conc={result['pass_concentration']}")
    _save("gate0", result)
    return result

def stage_gate1(panel, calib, t0, n_draws=config.N_BLOCK_DRAWS, force=False):
    if not force and (cached := _load("gate1")) is not None:
        return cached
    n, h = PRIMARY_CELL["n"], PRIMARY_CELL["h"]
    c = calib["c"][n]
    sig = signal_mod.build_signal(panel, n=n, c=c)
    ic_res = robustness.event_level_ic(panel, sig, h, side="both")
    _log(t0, f"gate1: real pooled event-level IC = {ic_res['ic']:.4f} (n_events={ic_res['n_events']})")
    null_res = robustness.ic_null_test(panel, sig, h, n_draws=n_draws, seed=7)
    _log(t0, f"gate1: IC null test p={null_res['p_value']:.4f} over {null_res['n_draws']} draws")
    redund = robustness.redundancy_regression(panel, sig, h)
    _log(t0, f"gate1: redundancy pooled R^2={redund['r2']:.4f} (n_obs={redund['n_obs']})")
    result = {
        "ic": ic_res["ic"], "n_events": ic_res["n_events"], "null_test": null_res,
        "redundancy": redund,
        "pass_ic": bool(np.isfinite(null_res["p_value"]) and null_res["p_value"] < config.IC_P_KILL
                        and abs(ic_res["ic"]) >= config.IC_ABS_KILL),
        "pass_redundancy": bool(np.isfinite(redund["r2"]) and redund["r2"] <= 0.5),
    }
    _save("gate1", result)
    return result

def stage_primary(panel, calib, t0, n_draws=config.N_BLOCK_DRAWS, force=False):
    if not force and (cached := _load("primary")) is not None:
        return cached
    n, pctl, h = PRIMARY_CELL["n"], PRIMARY_CELL["pctl"], PRIMARY_CELL["h"]
    c = calib["c"][n]
    sig = signal_mod.build_signal(panel, n=n, c=c)
    th_high, th_low = calib["theta_high"][n][pctl], calib["theta_low"][n][pctl]
    anti_th_high, anti_th_low = calib["anti_theta_high"][n][pctl], calib["anti_theta_low"][n][pctl]

    res = backtest.run_is_oos(panel, sig, th_high, th_low, h)
    _log(t0, f"primary: k={res['k']:.5f} achieved_IS_vol={res['achieved_vol_is']:.4f} "
           f"n_trades={len(res['trades'])} IS_sharpe={metrics.sharpe(res['returns_is']):.3f} "
           f"00S1_sharpe={metrics.sharpe(res['returns_oos1']):.3f}")

    donchian = twins.run_donchian_twin(panel, sig, h)
    _log(t0, f"primary: Donchian twin n_trades={len(donchian['trades'])} "
           f"IS_sharpe={metrics.sharpe(donchian['returns_is']):.3f} "
           f"00S1_sharpe={metrics.sharpe(donchian['returns_oos1']):.3f}")

    anti = twins.run_anti_twin(panel, sig, anti_th_high, anti_th_low, h)
    _log(t0, f"primary: anti-twin n_trades={len(anti['trades'])} "
           f"IS_sharpe={metrics.sharpe(anti['returns_is']):.3f} "
           f"00S1_sharpe={metrics.sharpe(anti['returns_oos1']):.3f}")

    null3 = battery.block_permuted_returns_null(panel, n, c, th_high, th_low, h, res["k"], n_draws=n_draws, seed=11)
    _log(t0, f"primary: null#3 (block-permuted returns) {null3['n_draws']} draws done")

    null4 = battery.randomized_entry_null(panel, sig, res["admitted"], h, res["k"], n_draws=n_draws, seed=13)
    _log(t0, f"primary: null#4 (randomized entry) {null4['n_draws']} draws done")

    tsmom_ret = tsmom.tsmom_returns(panel)
    common_idx = res["returns_full"].dropna().index.intersection(tsmom_ret.dropna().index)
    tsmom_corr = res["returns_full"].reindex(common_idx).corr(tsmom_ret.reindex(common_idx))
    tsmom_corr_oos1 = (res["returns_oos1"].dropna().index.intersection(tsmom_ret.dropna().index))
    tsmom_corr_oos1_val = res["returns_oos1"].reindex(tsmom_corr_oos1).corr(tsmom_ret.reindex(tsmom_corr_oos1))
    _log(t0, f"primary: corr(TSMOM) full={tsmom_corr:.3f} OOS1={tsmom_corr_oos1_val:.3f}")

    sign_full = metrics.sub_period_sign_consistency(res["returns_full"], config.IS_START, config.OOS1_END)
    concentration_full = metrics.pnl_quarter_concentration(res["returns_full"])
    _log(t0, f"primary: sub-period sign inconsistency={sign_full['n_inconsistent']}/4, "
           f"max-quarter PnL share={concentration_full:.3f}")

    result = {
        "primary": res, "donchian": donchian, "anti_twin": anti, "null3": null3, "null4": null4,
        "tsmom_corr_full": tsmom_corr, "tsmom_corr_oos1": tsmom_corr_oos1_val,
        "sign_consistency": sign_full, "pnl_quarter_concentration": concentration_full,
        "theta_high": th_high, "theta_low": th_low, "k": res["k"], "n": n, "c": c, "h": h, "pctl": pctl,
    }

```

```

    }
    _save("primary", result)
    return result

def stage_grid(panel, calib, t0, force=False):
    if not force and (cached := _load("grid")) is not None:
        return cached
    res = grid_mod.run_grid(panel, calib)
    primary_cell_id = f"n{PRIMARY_CELL['n']}_p{PRIMARY_CELL['pctl']}_h{PRIMARY_CELL['h']}"
    dsr_is = grid_mod.dsr_from_grid(res["table"], res["returns_by_cell"], primary_cell_id, period="is")
    dsr_oos = grid_mod.dsr_from_grid(res["table"], res["returns_by_cell"], primary_cell_id, period="oos1")
    neighborhood = grid_mod.neighborhood_isolation_check(res["table"], primary_cell_id, sharpe_col="is_sharpe")
    neighborhood_oos = grid_mod.neighborhood_isolation_check(res["table"], primary_cell_id, sharpe_col="oos1_sharpe")
    _log(t0, f"grid: DSR(IS) prob={dsr_is['dsr_prob']:.4f} excess={dsr_is['dsr_excess']:.4f}")
    _log(t0, f"grid: DSR(OOS1) prob={dsr_oos['dsr_prob']:.4f} excess={dsr_oos['dsr_excess']:.4f}")
    _log(t0, f"grid: neighborhood sign-majority IS={neighborhood['frac_same_sign']:.2f} "
            f"OOS1={neighborhood_oos['frac_same_sign']:.2f}")
    result = {"table": res["table"], "dsr_is": dsr_is, "dsr_oos1": dsr_oos,
            "neighborhood_is": neighborhood, "neighborhood_oos1": neighborhood_oos,
            "primary_cell_id": primary_cell_id}
    _save("grid", result)
    return result

def stage_oos2(calib, primary_result, t0, force=False):
    """Single confirmation run on the 16-country-ETF surface, using every
    parameter frozen from the primary-universe IS calibration: n, c, theta_i
    (recalibrated PER-ASSET on THIS universe's own IS window -- theta_i is
    explicitly an asset-specific threshold and the country ETFs are
    different assets, so a literal transplant of the primary universe's
    per-ticker theta_i values would be a category error; n, c, h, and k are
    transplanted verbatim, unchanged). Run exactly once."""
    if not force and (cached := _load("oos2")) is not None:
        return cached
    panel2 = data.load_secondary_panel()
    n, pctl, h = PRIMARY_CELL["n"], PRIMARY_CELL["pctl"], PRIMARY_CELL["h"]
    c = calib["c"][n]
    sig2 = signal_mod.build_signal(panel2, n=n, c=c)

    from . import signal as signal_mod2
    th_high2 = signal_mod2.calibrate_theta_multi(panel2, n=n, c=c, pctl=[pctl], side="high",
                                                n_draws=config.N_THETA_CALIB_DRAWS, seed=500)[pctl]
    th_low2 = signal_mod2.calibrate_theta_multi(panel2, n=n, c=c, pctl=[pctl], side="low",
                                                n_draws=config.N_THETA_CALIB_DRAWS, seed=600)[pctl]

    candidates = backtest.generate_candidates(panel2, sig2, th_high2, th_low2, h)
    long_c = [c_ for c_ in candidates if c_.direction == 1]
    short_c = [c_ for c_ in candidates if c_.direction == -1]

    k_frozen = primary_result["k"]
    admitted = backtest.admit_candidates(candidates)
    trades = backtest.assign_weights(panel2, admitted, k_frozen)
    rets = backtest.daily_returns(panel2, trades)

    donchian2 = twins.run_donchian_twin(panel2, sig2, h)

    result = {
        "n_long": len(long_c), "n_short": len(short_c),
        "conc_long": metrics.event_concentration(long_c, panel2.tickers),
        "conc_short": metrics.event_concentration(short_c, panel2.tickers),
        "n_trades": len(trades), "returns": rets,
        "sharpe_full": metrics.sharpe(rets), "ann_return": metrics.ann_return(rets),
        "ann_vol": metrics.ann_vol(rets),
        "donchian_sharpe": metrics.sharpe(donchian2["returns_full"]),
        "theta_high": th_high2, "theta_low": th_low2,
    }
    _log(t0, f"oos2: n_long={result['n_long']} n_short={result['n_short']} n_trades={result['n_trades']} "
            f"sharpe={result['sharpe_full']:.3f} donchian_sharpe={result['donchian_sharpe']:.3f}")
    _save("oos2", result)
    return result

def run(force=False, n_draws=config.N_BLOCK_DRAWS):
    t0 = time.time()
    calib = run_calibration.run()
    _log(t0, "calibration loaded")
    panel = data.load_primary_panel()

    gate0 = stage_gate0(panel, calib, t0, force=force)
    gate1 = stage_gate1(panel, calib, t0, n_draws=n_draws, force=force)
    primary = stage_primary(panel, calib, t0, n_draws=n_draws, force=force)

```

```
grid_res = stage_grid(panel, calib, t0, force=force)
oos2 = stage_oos2(calib, primary["primary"], t0, force=force)

_log(t0, "pipeline complete")
return {"gate0": gate0, "gate1": gate1, "primary": primary, "grid": grid_res, "oos2": oos2}

if __name__ == "__main__":
    run(force=True)
```

```

import numpy as np
import pandas as pd
import pytest

from research.dammluckan import signal, data, config

def _toy_panel(prices, dates=None, ticker="TST"):
    n = len(prices)
    if dates is None:
        dates = pd.bdate_range("2020-01-01", periods=n)
    close = pd.DataFrame({ticker: prices}, index=dates)
    vol = pd.DataFrame({ticker: [1_000_000.0] * n}, index=dates)
    return data.Panel(tickers=[ticker], raw_close=close, raw_open=close, high=close, low=close,
                      adj_close=close, adj_open=close, volume=vol)

def test_m_high_excludes_today():
    # A strictly increasing series: M_t at position t must be the max of the
    # PRECEDING n values only, so a new all-time high always satisfies E+.
    prices = np.arange(1.0, 51.0)
    panel = _toy_panel(prices)
    m = signal.rolling_max_excl_today(panel.raw_close, n=5)["TST"]
    # At position 10 (0-indexed), preceding 5 values are prices[5:10] = 6..10
    assert m.iloc[10] == prices[9] # max of prices[5:10] (0-indexed) == prices[9]
    # Today's own price must never be included: mutate prices[10] and confirm
    # m.iloc[10] (which depends only on prices[5:10]) is unaffected.
    prices2 = prices.copy()
    prices2[10] = 999.0
    panel2 = _toy_panel(prices2)
    m2 = signal.rolling_max_excl_today(panel2.raw_close, n=5)["TST"]
    assert m2.iloc[10] == m.iloc[10]

def test_no_lookahead_prefix_invariance():
    """Mutating price strictly AFTER date t must not change M_t/E+_t/O+_t
    computed at t -- the central causality guarantee of the whole signal."""
    rng = np.random.default_rng(0)
    prices = 100 * np.exp(np.cumsum(rng.normal(0, 0.01, 300)))
    panel_a = _toy_panel(prices)

    prices_b = prices.copy()
    prices_b[200:] = prices_b[200:] * 1.5 + rng.normal(0, 5, 100) # scramble everything from t=200 on
    panel_b = _toy_panel(prices_b)

    n, c = 60, 1.0
    band_vol_a = panel_a.band_vol.shift(1)
    band_vol_b = panel_b.band_vol.shift(1)

    m_a = signal.rolling_max_excl_today(panel_a.raw_close, n)["TST"]
    m_b = signal.rolling_max_excl_today(panel_b.raw_close, n)["TST"]
    assert np.allclose(m_a.iloc[:200], m_b.iloc[:200], equal_nan=True)

    o_a = signal.occupation_high(panel_a.raw_close, band_vol_a, n, c)["TST"]
    o_b = signal.occupation_high(panel_b.raw_close, band_vol_b, n, c)["TST"]
    # band_vol at t=199 depends on returns through t=199 only (shift(1) means
    # it uses data through t-1=198), unaffected by the t>=200 mutation.
    assert np.allclose(o_a.iloc[:199], o_b.iloc[:199], equal_nan=True)

def test_occupation_bounds():
    """0+ is always a fraction in [0, 1] wherever it's defined, and a
    perfectly flat price series has 0+ == 1 (every day sits at the ceiling)."""
    prices = np.full(200, 100.0)
    panel = _toy_panel(prices)
    band_vol = panel.band_vol.shift(1).fillna(0.0)
    o = signal.occupation_high(panel.raw_close, band_vol, n=50, c=1.0)["TST"].dropna()
    assert (o >= 0).all() and (o <= 1).all()
    assert np.allclose(o, 1.0)

def test_occupation_monotone_in_c():
    """Widening the band (larger c) can only ever increase or hold 0+ fixed,
    never decrease it -- more days qualify as the band widens."""
    rng = np.random.default_rng(1)
    prices = 100 * np.exp(np.cumsum(rng.normal(0, 0.01, 400)))
    panel = _toy_panel(prices)
    band_vol = panel.band_vol.shift(1)

```

```

o_narrow = signal.occupation_high(panel.raw_close, band_vol, n=60, c=0.5)["TST"]
o_wide = signal.occupation_high(panel.raw_close, band_vol, n=60, c=3.0)["TST"]
valid = o_narrow.notna() & o_wide.notna()
assert (o_wide[valid] >= o_narrow[valid] - 1e-12).all()

def test_record_event_matches_direct_comparison():
    rng = np.random.default_rng(2)
    prices = 100 * np.exp(np.cumsum(rng.normal(0, 0.01, 250)))
    panel = _toy_panel(prices)
    n = 40
    m = signal.rolling_max_excl_today(panel.raw_close, n)
    e = signal.record_events_high(panel.raw_close, m)["TST"]
    px = panel.raw_close["TST"]
    for t in range(n + 1, len(prices)):
        expected = float(px.iloc[t] > m["TST"].iloc[t])
        assert e.iloc[t] == expected

```

```

import numpy as np
import pandas as pd
import pytest

from research.dammluckan import backtest, config, data
from research.dammluckan.portfolio import CandidateTrade, Trade

def test_compute_exit_time_stop_when_no_opposite_event():
    T = 30
    opp = np.zeros(T)
    exit_pos, reason = backtest.compute_exit(entry_pos=5, h=10, opp_events=opp, T=T)
    assert exit_pos == 15
    assert reason == "time_stop"

def test_compute_exit_opposite_record_wins_when_earlier():
    T = 30
    opp = np.zeros(T)
    opp[8] = 1.0 # decision date 8 -> execution 9, strictly before time-stop exec 15
    exit_pos, reason = backtest.compute_exit(entry_pos=5, h=10, opp_events=opp, T=T)
    assert exit_pos == 9
    assert reason == "opposite_record"

def test_compute_exit_time_stop_wins_on_tie():
    T = 30
    opp = np.zeros(T)
    opp[13] = 1.0 # decision 13 -> execution 14 == entry(5)+h-1(9)? check boundary precisely
    # entry=5, h=10 -> exec_time_stop = 15. scan window is [5, 5+10-1]=[5,14).
    # opp[13]=1 -> candidate_exit = 13+1 = 14 < 15 -> should be admitted as opposite_record.
    exit_pos, reason = backtest.compute_exit(entry_pos=5, h=10, opp_events=opp, T=T)
    assert exit_pos == 14
    assert reason == "opposite_record"
    # Now place the opposite event exactly at the scan boundary (excluded):
    opp2 = np.zeros(T)
    opp2[14] = 1.0 # outside [5,14) scan window -> ignored, time-stop wins
    exit_pos2, reason2 = backtest.compute_exit(entry_pos=5, h=10, opp_events=opp2, T=T)
    assert exit_pos2 == 15
    assert reason2 == "time_stop"

def test_generate_candidates_donchian_twin_theta_minus_inf_not_treated_as_unavailable():
    """Regression test: theta=-inf (the Donchian twin's "always admit" gate)
    must NOT be treated the same as theta=NaN ("no threshold available").
    np.isfinite(-inf) is False, so a naive `if not isfinite(theta): skip`
    guard silently drops every ticker for the Donchian twin -- this must not
    happen."""
    from research.dammluckan import signal as signal_mod

    rng = np.random.default_rng(3)
    n = 300
    dates = pd.bdate_range("2020-01-01", periods=n)
    prices = 100 * np.exp(np.cumsum(rng.normal(0, 0.01, n)))
    close = pd.DataFrame({"TST": prices}, index=dates)
    vol = pd.DataFrame({"TST": [1_000_000.0] * n}, index=dates)
    panel = data.Panel(tickers=["TST"], raw_close=close, raw_open=close, high=close, low=close,
                       adj_close=close, adj_open=close, volume=vol)
    sig = signal_mod.build_signal(panel, n=40, c=1.0)

    gated_theta = pd.Series({"TST": 0.9}) # a real, restrictive threshold
    gated = backtest.generate_candidates(panel, sig, gated_theta, gated_theta, h=10)

    neg_inf_theta = pd.Series({"TST": -np.inf}) # Donchian twin: always admit
    ungated = backtest.generate_candidates(panel, sig, neg_inf_theta, neg_inf_theta, h=10)

    assert len(ungated) > 0
    assert len(ungated) >= len(gated)

def test_compute_exit_clips_at_end_of_data():
    T = 12
    opp = np.zeros(T)
    exit_pos, reason = backtest.compute_exit(entry_pos=5, h=10, opp_events=opp, T=T)
    assert exit_pos == T - 1
    assert reason == "time_stop"

```

```

def _candidate(ticker, direction, entry_pos, exit_pos, o_value=0.5):
    return CandidateTrade(
        ticker=ticker, direction=direction, decision_date=pd.Timestamp("2020-01-01"),
        entry_date=pd.Timestamp("2020-01-02"), entry_pos=entry_pos,
        exit_date=pd.Timestamp("2020-01-03"), exit_pos=exit_pos, exit_reason="time_stop",
        entry_price=100.0, exit_price=101.0, o_value=o_value, entry_adv=1e9, exit_adv=1e9,
    )

def test_admit_respects_max_concurrent():
    # 15 candidates all overlapping [entry=0, exit=20], different tickers ->
    # only MAX_CONCURRENT should be admitted.
    candidates = [_candidate(f"T{i}", 1, 0, 20) for i in range(15)]
    admitted = backtest.admit_candidates(candidates, max_concurrent=12)
    assert len(admitted) == 12

def test_admit_prioritizes_higher_occupation_on_contention():
    candidates = [_candidate(f"T{i}", 1, 0, 20, o_value=float(i)) for i in range(15)]
    admitted = backtest.admit_candidates(candidates, max_concurrent=12)
    admitted_tickers = {c.ticker for c in admitted}
    # highest o_value candidates are T14..T3 (12 of them); T0,T1,T2 should be dropped
    assert admitted_tickers == {f"T{i}" for i in range(3, 15)}

def test_admit_ignores_new_event_while_asset_has_open_position():
    c1 = _candidate("SPY", 1, entry_pos=0, exit_pos=20)
    c2 = _candidate("SPY", 1, entry_pos=5, exit_pos=25) # fires while c1 still open
    admitted = backtest.admit_candidates([c1, c2], max_concurrent=12)
    assert len(admitted) == 1
    assert admitted[0] is c1

def test_admit_allows_reentry_after_exit():
    c1 = _candidate("SPY", 1, entry_pos=0, exit_pos=10)
    c2 = _candidate("SPY", 1, entry_pos=10, exit_pos=20) # entry == prior exit: slot freed in time
    admitted = backtest.admit_candidates([c1, c2], max_concurrent=12)
    assert len(admitted) == 2

def _toy_panel_with_open(prices_close, prices_open, dates=None, ticker="TST"):
    n = len(prices_close)
    if dates is None:
        dates = pd.bdate_range("2020-01-01", periods=n)
    close = pd.DataFrame({ticker: prices_close}, index=dates)
    open_ = pd.DataFrame({ticker: prices_open}, index=dates)
    vol = pd.DataFrame({ticker: [1_000_000.0] * n}, index=dates)
    return data.Panel(tickers=[ticker], raw_close=close, raw_open=open_, high=close, low=close,
        adj_close=close, adj_open=open_, volume=vol)

def test_daily_returns_hand_computed_three_day_hold():
    # 4 trading days: entry executes day 1 (open), held through day 2
    # (close-to-close), exits day 3 (open). Hand-computed leg-by-leg.
    closes = [100.0, 102.0, 105.0, 103.0]
    opens = [100.0, 101.0, 103.0, 104.0]
    panel = _toy_panel_with_open(closes, opens)
    trade = Trade(
        ticker="TST", direction=1, decision_date=panel.dates[0], entry_date=panel.dates[1],
        entry_pos=1, exit_date=panel.dates[3], exit_pos=3, exit_reason="time_stop",
        entry_price=opens[1], exit_price=opens[3], o_value=0.5, entry_adv=1e9, exit_adv=1e9,
        weight=0.20, sigma_hat=0.15, gross_return=(opens[3] / opens[1] - 1.0), cost_frac=0.001,
    )
    rets = backtest.daily_returns(panel, [trade])
    w = 0.20
    # day1 (entry, pos=1): open->close = 101 -> 102
    expected_day1 = w * (closes[1] / opens[1] - 1.0) - w * 0.001
    # day2 (held, pos=2): close[1]->close[2] = 102 -> 105
    expected_day2 = w * (closes[2] / closes[1] - 1.0)
    # day3 (exit, pos=3): close[2]->open[3] = 105 -> 104
    expected_day3 = w * (opens[3] / closes[2] - 1.0)
    assert abs(rets.iloc[0]) < 1e-12
    assert abs(rets.iloc[1] - expected_day1) < 1e-12
    assert abs(rets.iloc[2] - expected_day2) < 1e-12
    assert abs(rets.iloc[3] - expected_day3) < 1e-12

def test_solve_k_for_target_vol_converges_when_gross_cap_binds():
    """Regression test: a single-shot linear rescale (k_final = k0 *
    target/vol(k0)) overshoots when the 200% gross cap binds, because the
    k->vol relationship stops being linear once positions get haircut.
    _solve_k_for_target_vol must iterate to convergence instead. Construct a

```

```

scenario with moderate cap pressure (staggered entries, realistic vol,
so several -- not all, not just one -- positions get haircut) and show
the iterative solve lands materially closer to the target than the
single-shot linear rescale it replaced."""
n = 550
dates = pd.bdate_range("2020-01-01", periods=n)
rng = np.random.default_rng(5)
closes = pd.DataFrame(
    {f"T{i}": 100 * np.exp(np.cumsum(rng.normal(0, 0.012, n))) for i in range(20)}, index=dates
)
vol = pd.DataFrame(1_000_000.0, index=dates, columns=closes.columns)
panel = data.Panel(tickers=list(closes.columns), raw_close=closes, raw_open=closes, high=closes,
                    low=closes, adj_close=closes, adj_open=closes, volume=vol)
# Staggered, long-overlapping windows -> the cap binds for a sustained
# stretch across many (not all) names, closer to the real Donchian-twin
# dynamics that exposed the original single-shot bug.
admitted = [_candidate(f"T{i}", 1, entry_pos=20 + 5 * i, exit_pos=350 + 5 * i) for i in range(20)]

target_vol = 0.08
k_final, _ = backtest.solve_k_for_target_vol(
    panel, admitted, is_start=dates[0], is_end=dates[-1], target_vol=target_vol, gross_cap=config.GROSS_CAP,
)
achieved_vol_iterative = backtest.annualized_vol(
    backtest.daily_returns(panel, backtest.assign_weights(panel, admitted, k_final, gross_cap=config.GROSS_CAP))
    .loc[dates[0]:dates[-1]]
)

# The single-shot approach the fix replaced: one linear rescale from k0.
trades0 = backtest.assign_weights(panel, admitted, target_vol, gross_cap=config.GROSS_CAP)
vol_at_k0 = backtest.annualized_vol(backtest.daily_returns(panel, trades0).loc[dates[0]:dates[-1]])
k_single_shot = target_vol * (target_vol / vol_at_k0)
achieved_vol_single_shot = backtest.annualized_vol(
    backtest.daily_returns(panel, backtest.assign_weights(panel, admitted, k_single_shot,
    gross_cap=config.GROSS_CAP))
    .loc[dates[0]:dates[-1]]
)

err_iterative = abs(achieved_vol_iterative - target_vol) / target_vol
err_single_shot = abs(achieved_vol_single_shot - target_vol) / target_vol
assert err_iterative < 0.05 # iterative solve lands within 5% of target
assert err_iterative < err_single_shot # and strictly beats the single-shot approach it replaced

def test_assign_weights_gross_cap_haircut():
    # Many simultaneous candidates with huge raw weight (tiny sigma_hat) must
    # be haircut so total gross never exceeds config.GROSS_CAP.
    n = 200
    dates = pd.bdate_range("2020-01-01", periods=n)
    closes = pd.DataFrame({f"T{i}": np.full(n, 100.0) + np.cumsum(np.random.default_rng(i).normal(0, 0.01, n))
                           for i in range(20)}, index=dates)
    opens = closes.copy()
    vol = pd.DataFrame(1_000_000.0, index=dates, columns=closes.columns)
    panel = data.Panel(tickers=list(closes.columns), raw_close=closes, raw_open=opens, high=closes, low=closes,
                        adj_close=closes, adj_open=opens, volume=vol)
    candidates = [_candidate(f"T{i}", 1, entry_pos=100, exit_pos=110) for i in range(20)]
    trades = backtest.assign_weights(panel, candidates, k=10.0, gross_cap=config.GROSS_CAP) # huge k
    gross = sum(abs(t.weight) for t in trades)
    assert gross <= config.GROSS_CAP + 1e-9

```



```
import numpy as np
from research.dammluckan import costs

def test_half_spread_bucket_boundaries():
    # Exact bucket boundaries from config.ADV_BUCKETS, hand-checked.
    assert costs.half_spread_bps(500e6) == 0.5
    assert costs.half_spread_bps(499_999_999) == 1.0
    assert costs.half_spread_bps(200e6) == 1.0
    assert costs.half_spread_bps(100e6) == 1.5
    assert costs.half_spread_bps(50e6) == 2.5
    assert costs.half_spread_bps(20e6) == 4.0
    assert costs.half_spread_bps(19_999_999) == 7.0
    assert costs.half_spread_bps(0.0) == 7.0

def test_round_trip_adds_commission():
    assert costs.round_trip_cost_bps(500e6) == 3.0 + 0.5
    assert costs.round_trip_cost_bps(0.0) == 3.0 + 7.0

def test_cost_fraction_hand_computed():
    # Hand-computed, independent of the production formula: a $600M-ADV name
    # costs 3.5bp round trip = 0.00035 of notional.
    frac = costs.cost_fraction(600e6)
    assert abs(frac - 0.00035) < 1e-12

def test_vectorized_matches_scalar():
    advs = np.array([1e9, 300e6, 75e6, 25e6, 5e6])
    vec = costs.half_spread_bps(advs)
    scalar = np.array([costs.half_spread_bps(float(a)) for a in advs])
    assert np.allclose(vec, scalar)
```

```

import numpy as np
import pandas as pd
import pytest

from research.dammluckan import metrics

def test_sharpe_hand_computed():
    r = pd.Series([0.01, -0.01, 0.02, 0.0, 0.01])
    expected = r.mean() / r.std(ddof=1) * np.sqrt(252)
    assert abs(metrics.sharpe(r) - expected) < 1e-12

def test_expected_max_sharpe_requires_multiple_trials():
    with pytest.raises(ValueError):
        metrics.expected_max_sharpe([0.1])

def test_expected_max_sharpe_increases_with_trial_count():
    rng = np.random.default_rng(0)
    trials_small = rng.normal(0, 0.05, 5)
    trials_large = rng.normal(0, 0.05, 500)
    small = metrics.expected_max_sharpe(trials_small)
    large = metrics.expected_max_sharpe(trials_large)
    assert large > small # more trials -> higher expected max under the null

def test_dsr_clear_outlier_scores_high():
    rng = np.random.default_rng(1)
    trials = rng.normal(0, 0.02, 50)
    sr_hat = 0.15 # far above the pack
    out = metrics.deflated_sharpe_ratio(sr_hat, n_obs=1000, sr_trials=trials)
    assert out["dsr_prob"] > 0.99

def test_dsr_typical_trial_scores_low():
    rng = np.random.default_rng(2)
    trials = rng.normal(0, 0.02, 50)
    sr_hat = float(np.median(trials))
    out = metrics.deflated_sharpe_ratio(sr_hat, n_obs=1000, sr_trials=trials)
    assert out["dsr_prob"] < 0.9

def test_pnl_quarter_concentration_single_quarter():
    dates = pd.bdate_range("2020-01-01", periods=300)
    r = pd.Series(0.0, index=dates)
    r.iloc[10:15] = 0.05 # all PnL concentrated in one quarter
    share = metrics.pnl_quarter_concentration(r)
    assert share > 0.99

def test_pnl_quarter_concentration_evenly_spread():
    dates = pd.bdate_range("2020-01-01", periods=1000)
    r = pd.Series(0.0001, index=dates) # flat positive drip, evenly spread
    share = metrics.pnl_quarter_concentration(r)
    assert share < 0.2

def test_sub_period_sign_consistency_all_positive():
    dates = pd.bdate_range("2020-01-01", periods=1000)
    r = pd.Series(0.0005, index=dates)
    res = metrics.sub_period_sign_consistency(r, dates[0], dates[-1], n_periods=4)
    assert res["n_inconsistent"] == 0

def test_event_concentration_flags_dominant_asset():
    from collections import namedtuple
    T = namedtuple("T", ["ticker"])
    trades = [T("SPY")] * 60 + [T("TLT")] * 40
    res = metrics.event_concentration(trades, ["SPY", "TLT", "GLD"])
    assert res["max_ticker"] == "SPY"
    assert abs(res["max_share"] - 0.6) < 1e-12

```